AI Orchestrator

A self-hosted platform for building, versioning, running, evaluating and governing AI agents and multi-step workflows with your team.

Project overview with screenshots of every page
Repository: AbdullKarimHamidi/AI-Orchestrator · branch claude/project-thread-qm6vbh
October 2026

What the project does

AI Orchestrator lets a team turn large language models into dependable, governed services. Developers define **agents**: a system prompt, a model with fallbacks, tools, knowledge bases, memory and limits. They test agents in a console, publish immutable versions, and combine agents into **workflows**. Operators and applications run them from the UI or the REST API. Every run is queued in MongoDB and executed by a separate **worker** process, so runs survive restarts and are streamed live, with every model call, tool call and cost recorded. **Evaluations** check quality before a release, and **analytics, budgets, plans, roles and the audit log** keep cost and access under control.

The app runs fully in **demo mode** without paying for any service. A clearly labelled Demo provider returns synthetic "[DEMO]" output at $0, and that is what the screenshots in this document show. Real OpenAI, Anthropic, Gemini or OpenAI-compatible models are used as soon as you add a key on the Models page.

Architecture

Component	Technology	Role
Web app	Next.js 16, React 19, TanStack Query, Zod, Tailwind, shadcn/ui	All pages. It calls the API through a same-origin proxy and validates every response.
API	FastAPI, Pydantic, PyMongo async	Authentication, role checks, validation, and business logic in services and repositories.
Worker	Python asyncio	Claims queued runs with leases and executes agents and workflows. It also ingests knowledge documents.
Database	MongoDB Atlas (cluster0.rxxertq.mongodb.net, database `AI`)	All data, and the durable run queue.
Production	Docker Compose, Caddy (automatic HTTPS), backup and restore scripts, GitHub Actions CI	Deployment and operations.

Main features

Agents with versioning
Draft, then publish an immutable version. Pause, resume, retire, roll back and compare versions. Each agent has a primary model and up to three fallbacks, parameters, budgets, input and output schemas and a test console.

Workflows
Chain agents, tools, knowledge search, templates and conditions with true/false routing. Workflows have draft and published versions and run durably in the worker with per-step live events.

Durable execution
The run queue lives in MongoDB: atomic leases, heartbeats, crash recovery, timeouts, cancellation, idempotency keys and Server-Sent Events streaming.

Model gateway
OpenAI, Anthropic, Gemini and OpenAI-compatible providers, with encrypted keys, health checks, model allowlists, usage and cost tracking, and a free Demo provider.

Tools and MCP
HTTP tools and tools imported from MCP servers, with JSON Schema validation, encrypted secret headers, SSRF protection and a per-agent allowlist.

Knowledge (RAG)
Upload documents; the worker chunks and embeds them; answers cite the passages they used.

Memory
Per-agent, per-user long-term memory with expiry, automatic conversation saving and deletion requests.

Evaluations
Datasets, scored evaluation runs and case-by-case comparison of two runs before you publish.

Teams and security
Workspaces, five roles enforced in the API, invitations, API keys, audit log, CSRF protection, rate limits and Argon2id password hashing.

Cost control
A monthly budget, per-run cost caps, plan quotas for runs and seats, analytics and monthly statements.

What was added in this update

Before this update, the Workflows, Memory, Evaluations and Billing pages only said "In development", and Analytics was partial. All of them are now working features with backend APIs, worker execution, tests and the pages shown in this document:

- **Workflows**: a visual and JSON step editor, routing with conditions, strict templates, a loop guard, versions and archiving. Workflow runs use the same worker, events and run inspector as agent runs.
- **Evaluations**: datasets with bulk import, evaluation runs with six scorers, a live pass rate and a comparison view.
- **Memory**: scoped memories added to agent prompts, automatic conversation saving, expiry and "forget" deletion requests.
- **Billing**: plans with seat and run quotas enforced by the API, and monthly statements with CSV export.
- **Analytics**: a daily runs chart and a breakdown per agent and workflow.
- **Database**: the env files now point at the project's MongoDB Atlas cluster (database `AI`).

Checks: all 90 backend tests pass, as do the 38 frontend unit tests, type checking, lint and the production build.

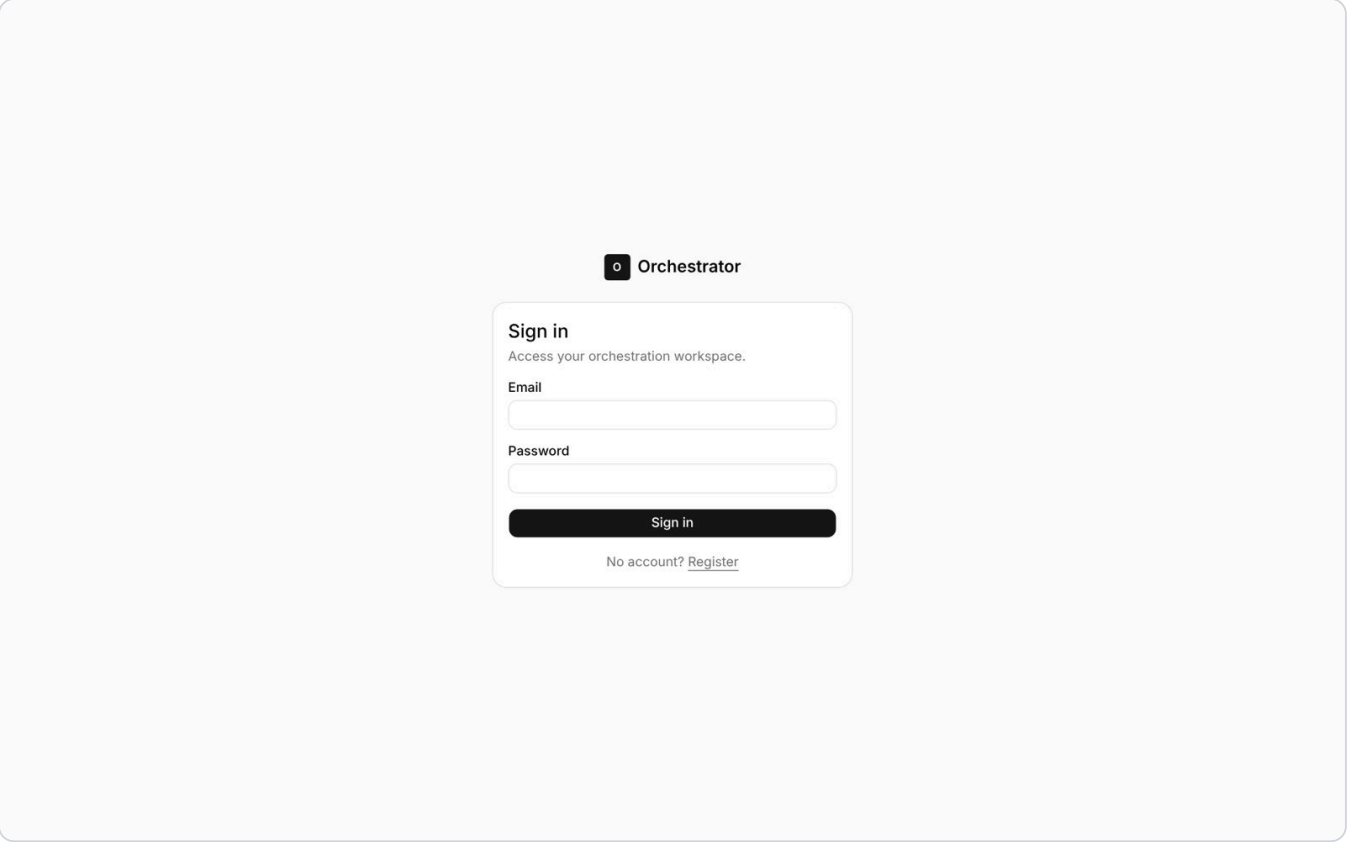
Pages in this document

1. **Getting started**: Sign in, Create an account, Accept an invitation, Dashboard
2. **Build: agents**: Agents, New agent, Agent overview, Edit agent, Test console, Versions
3. **Build: workflows (new)**: Workflows, New workflow, Workflow overview and run, Edit workflow, Workflow runs
4. **Build: tools, integrations, knowledge and memory**: Tools, New tool, Tool detail and test console, Integrations, MCP import, Knowledge, Knowledge bases, Knowledge base detail, Documents, Memory (new)
5. **Operate**: Runs, Run inspector (agent), Run inspector (workflow), Models, Analytics (completed), Evaluations (new), Evaluation datasets, Evaluation runs, Compare evaluations
6. **Govern**: Security, Users, Roles, Audit log, Settings, Workspace settings, Account, API keys, Billing and plan (new), API documentation

Getting started

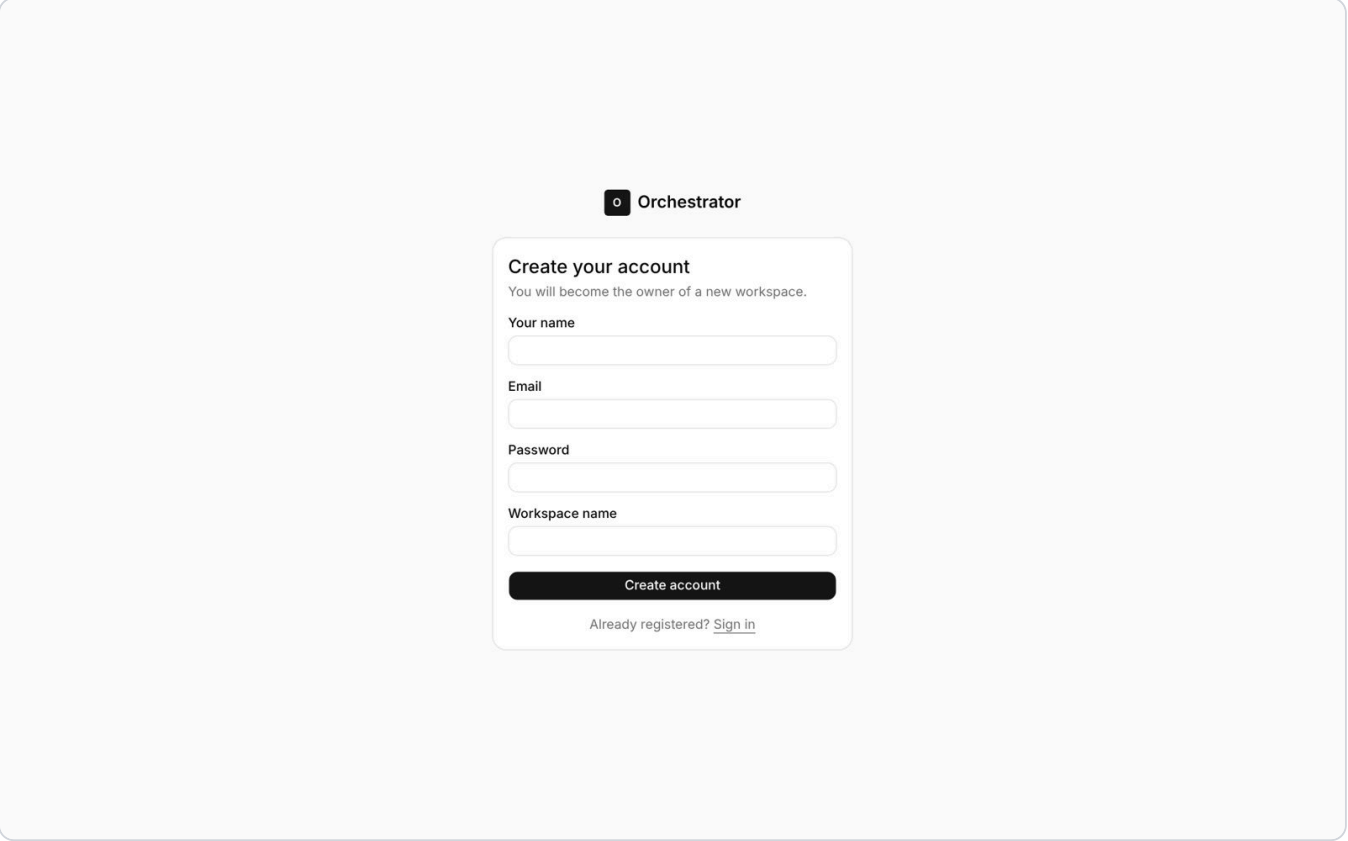
1. Sign in `/login`

Email and password sign-in. Sessions are HttpOnly cookies with CSRF protection; passwords are hashed with Argon2id and sign-in attempts are rate limited.



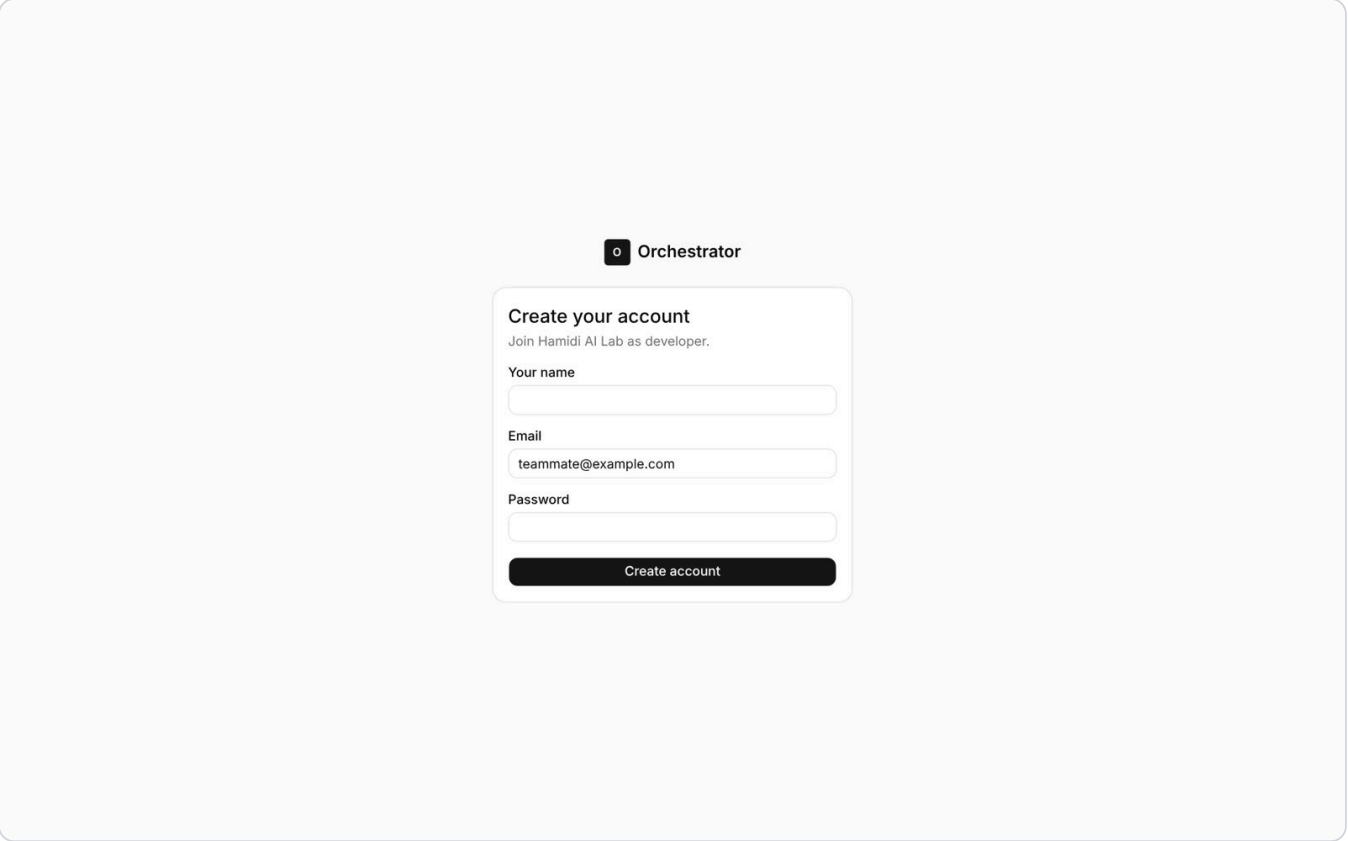
2. Create an account `/register`

Self-service registration creates the user, a first workspace and the owner membership in one transaction. Admins can close registration (ALLOW_REGISTRATION=false) so new people join only by invitation.



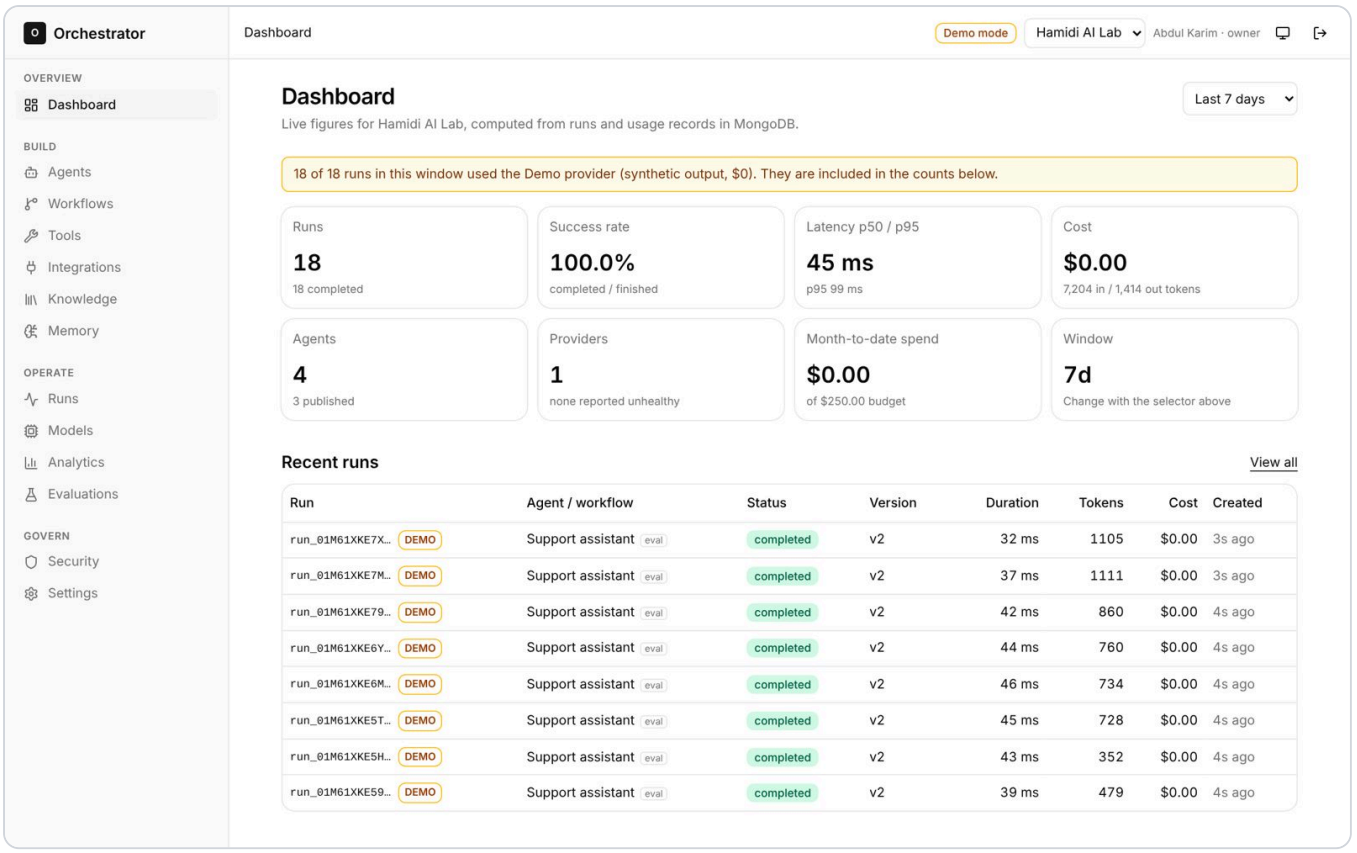
3. Accept an invitation `/invite/{token}`

Single-use invitation link for a given email and role. The invitee sees the workspace and role before joining, and can register or sign in.



4. Dashboard `/dashboard`

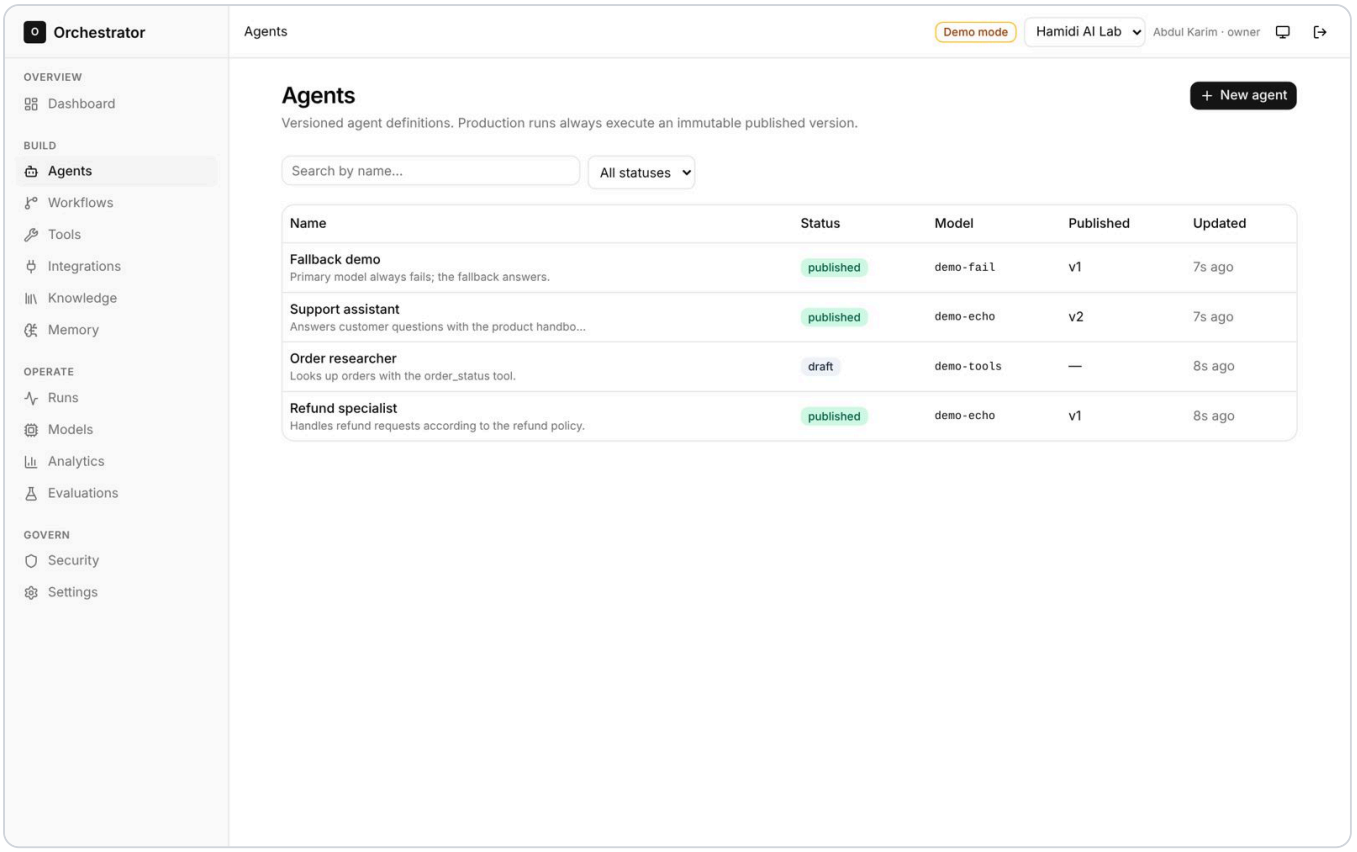
Workspace overview: agents, runs by status, success rate, p50/p95 latency, tokens and cost, month-to-date spend against the budget, provider health and the latest runs. It refreshes every 15 seconds.



Build: agents

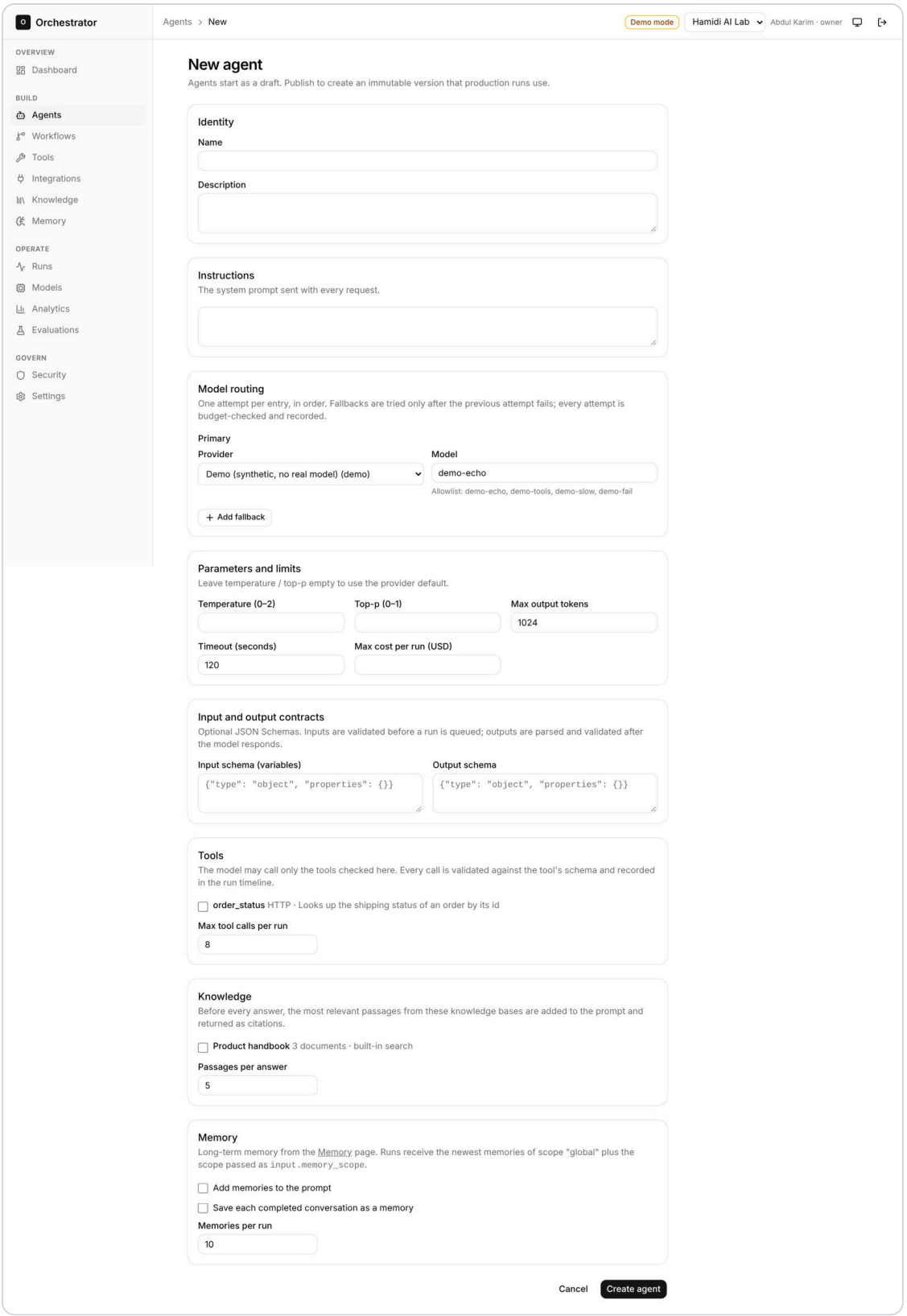
5. Agents `/agents`

All agents in the workspace with status (draft, published, paused, retired), model and last update. You can search them and filter by status.



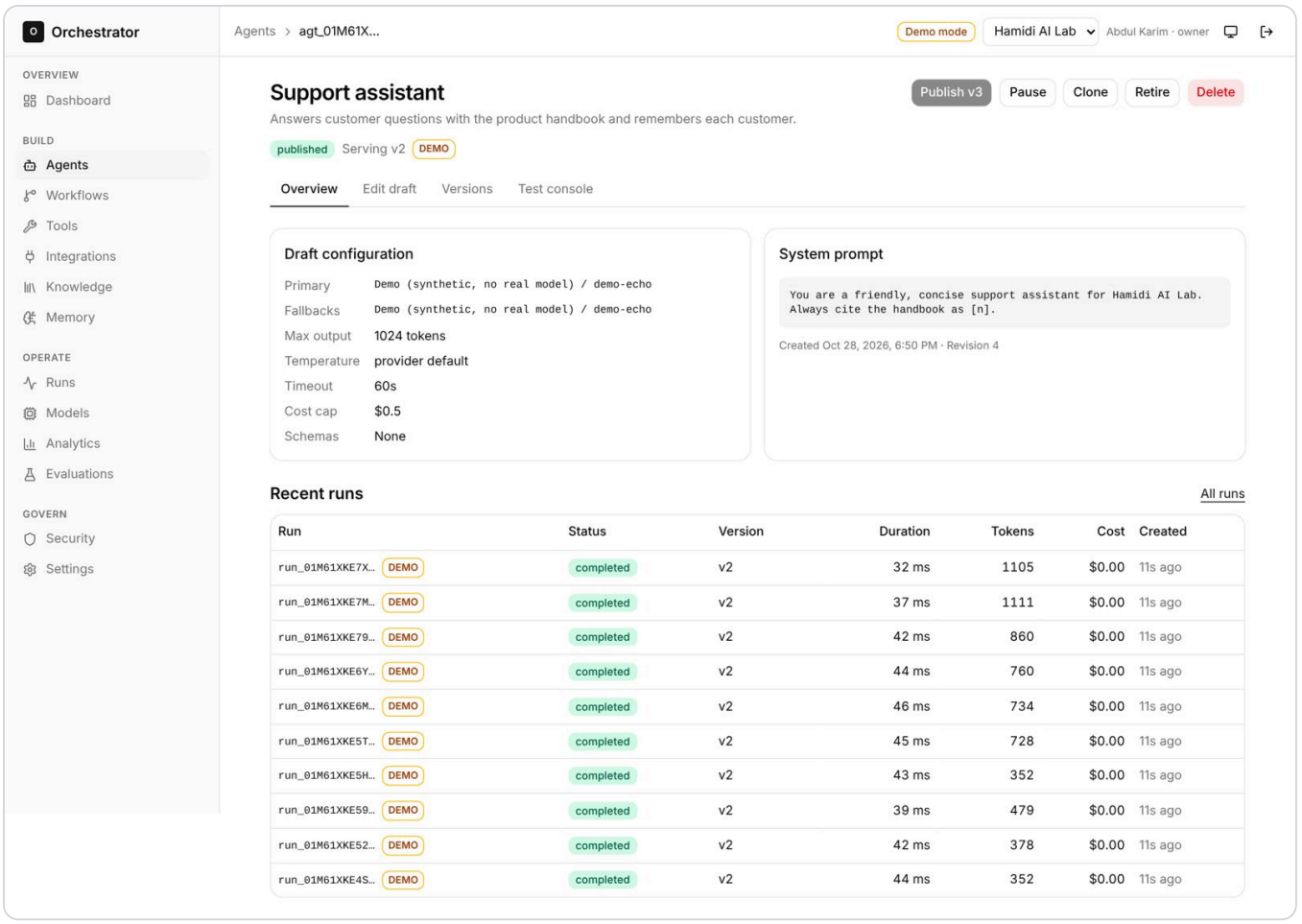
6. New agent `/agents/new`

Defines an agent: system prompt, primary model plus up to three fallbacks, parameters, limits (timeout, cost cap, tool calls), JSON Schemas for input and output, allowed tools, knowledge bases and memory.



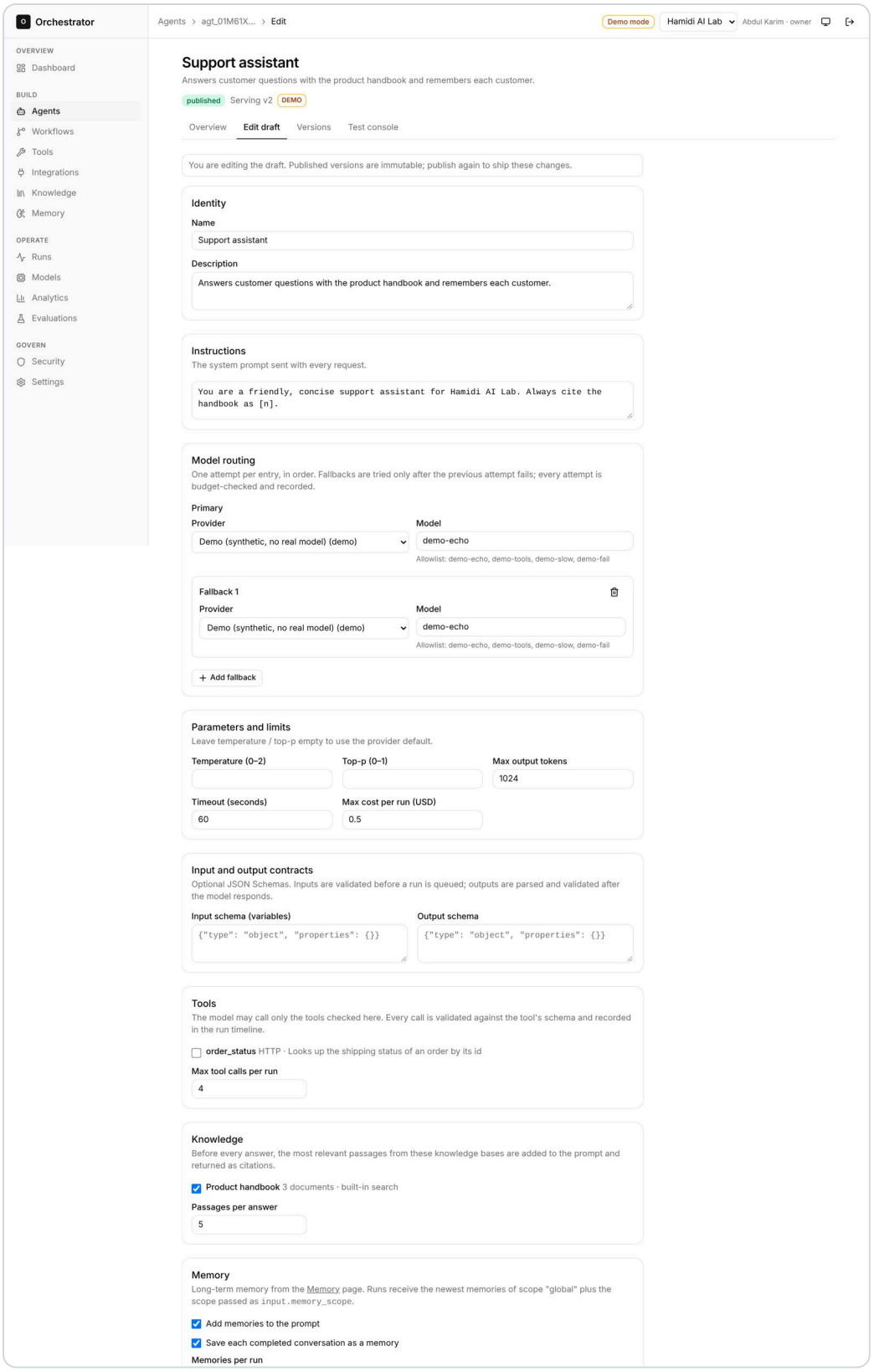
7. Agent overview `/agents/{id}`

Status, published version, configuration summary and lifecycle actions: publish, pause, resume, retire, clone and delete.



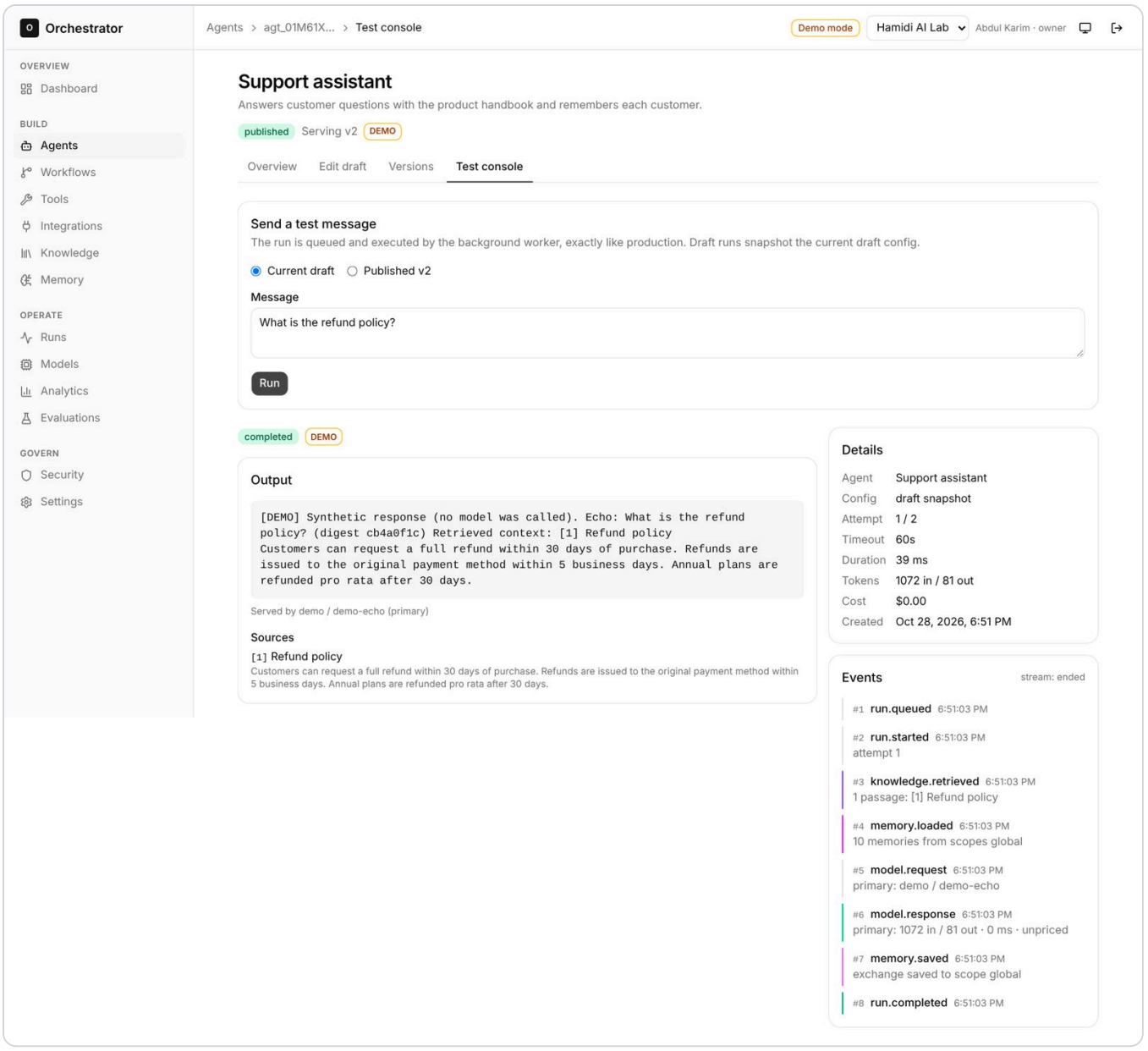
8. Edit agent `/agents/{id}/edit`

Edits the draft only. Published versions are immutable, so production behaviour changes only when you publish again. Optimistic concurrency stops two people from overwriting each other. This is where you switch on the new Memory options.



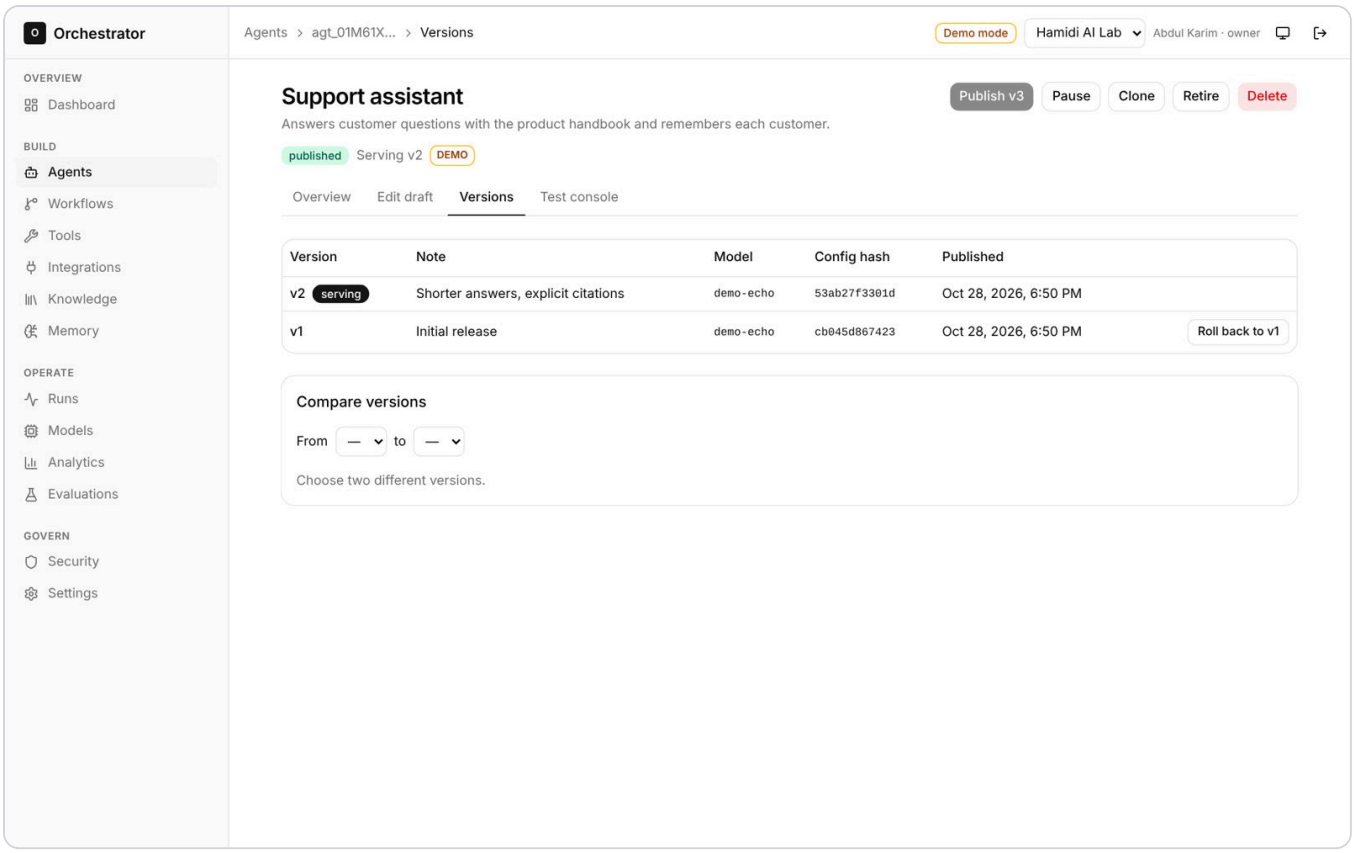
9. Test console `/agents/{id}/test`

Runs the draft or a published version and streams the run's events live (Server-Sent Events): model requests and responses, tool calls, knowledge retrieval, memory and the final output.



10. Versions `/agents/{id}/versions`

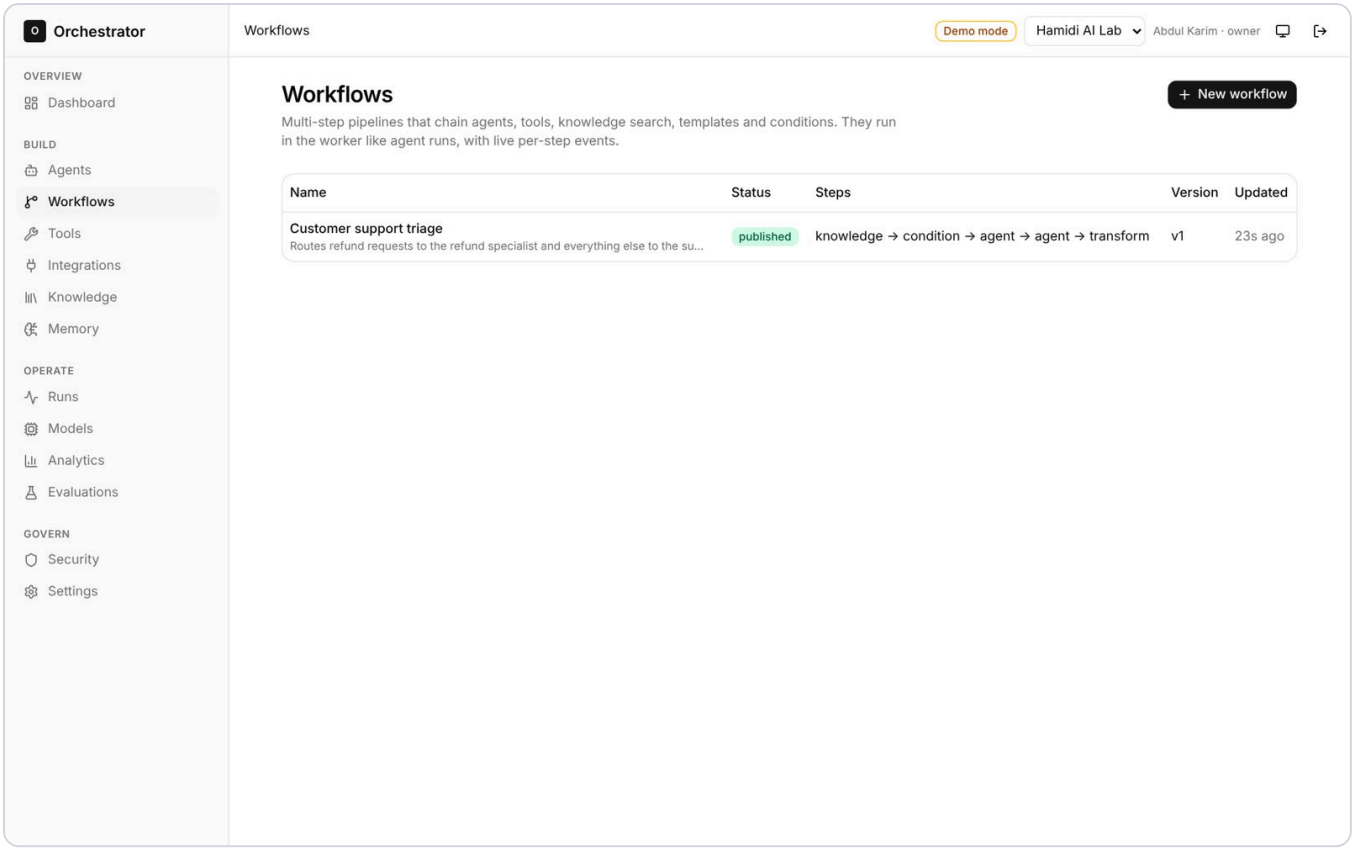
Immutable version history with notes and config hashes, a field-by-field diff between any two versions, and rollback.



Build: workflows (new)

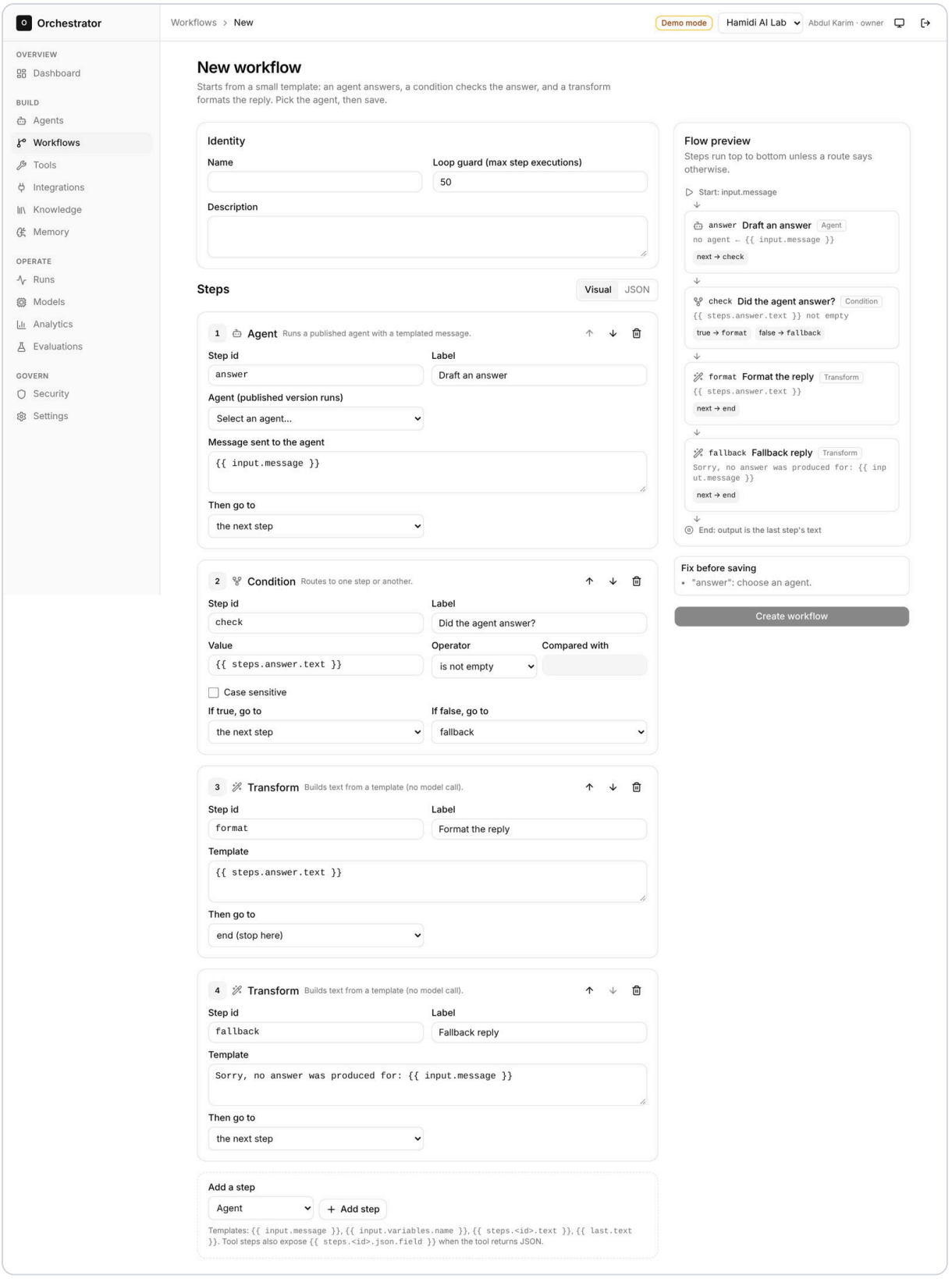
11. Workflows `/workflows`

Multi-step pipelines with their status, the step types they use and the published version.



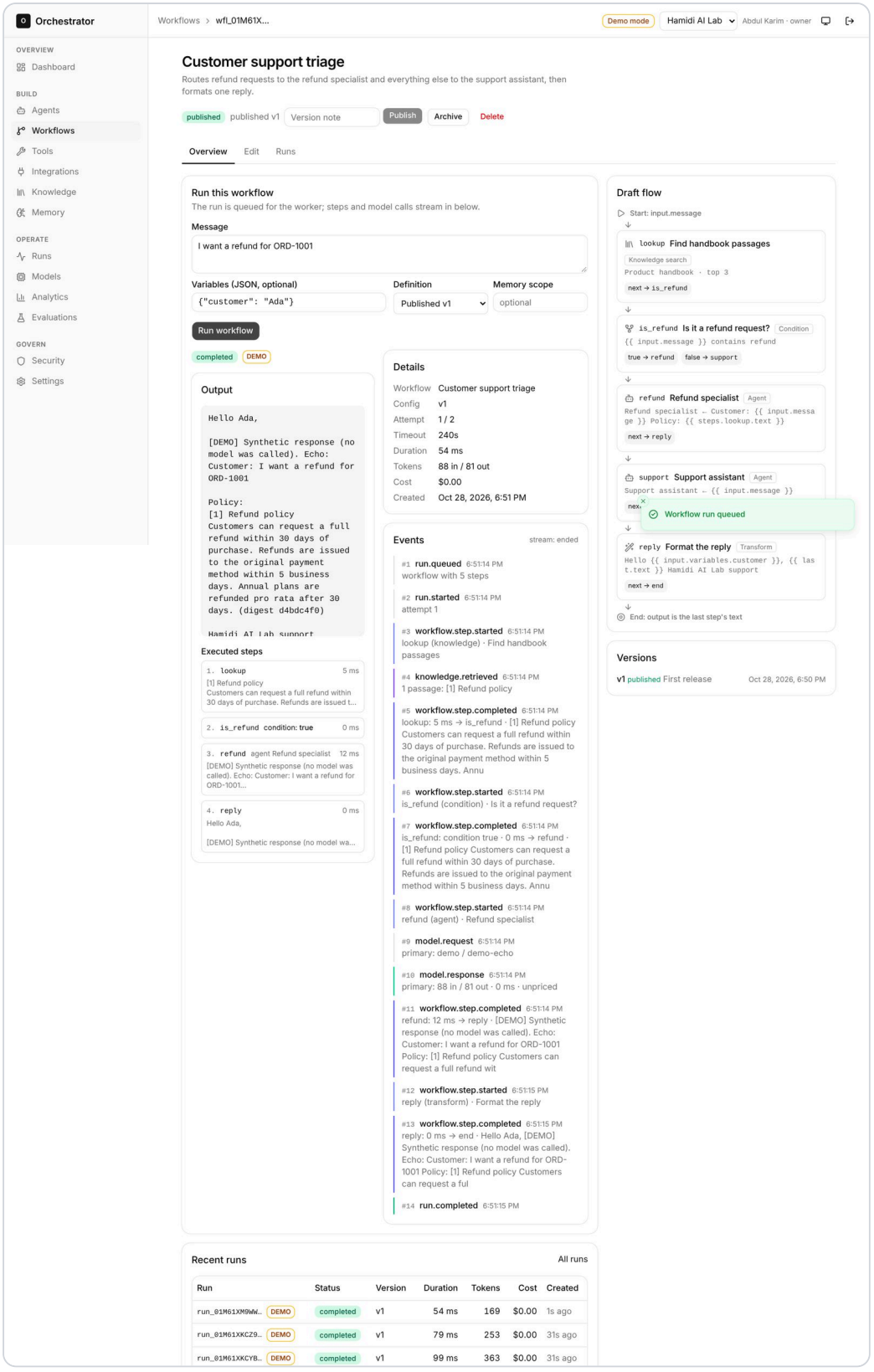
12. New workflow `/workflows/new`

The visual step editor opens with a starter template. Step types: Agent (runs a published agent), Tool, Knowledge search, Transform (template) and Condition (routes to a true or false branch). The flow preview on the right updates as you edit, and problems are listed before you save.



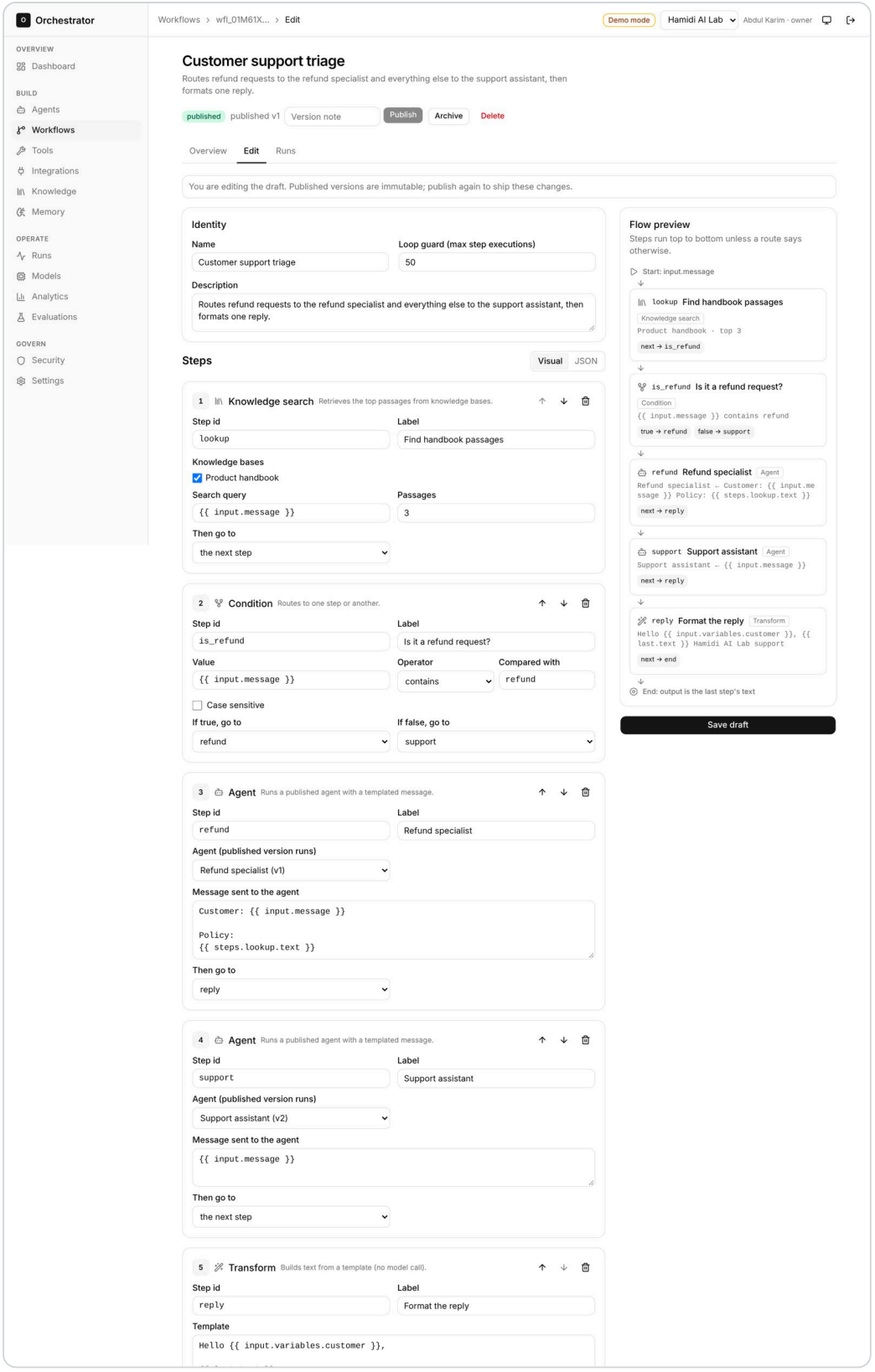
13. Workflow overview and run `/workflows/{id}`

Run the workflow with a message, optional variables and a memory scope. Here the condition detected a refund request, so the refund specialist answered, using the passages found by the knowledge step, and the final transform formatted the reply. Each executed step, its output and the live event timeline are shown, together with the flow diagram, the versions and recent runs.



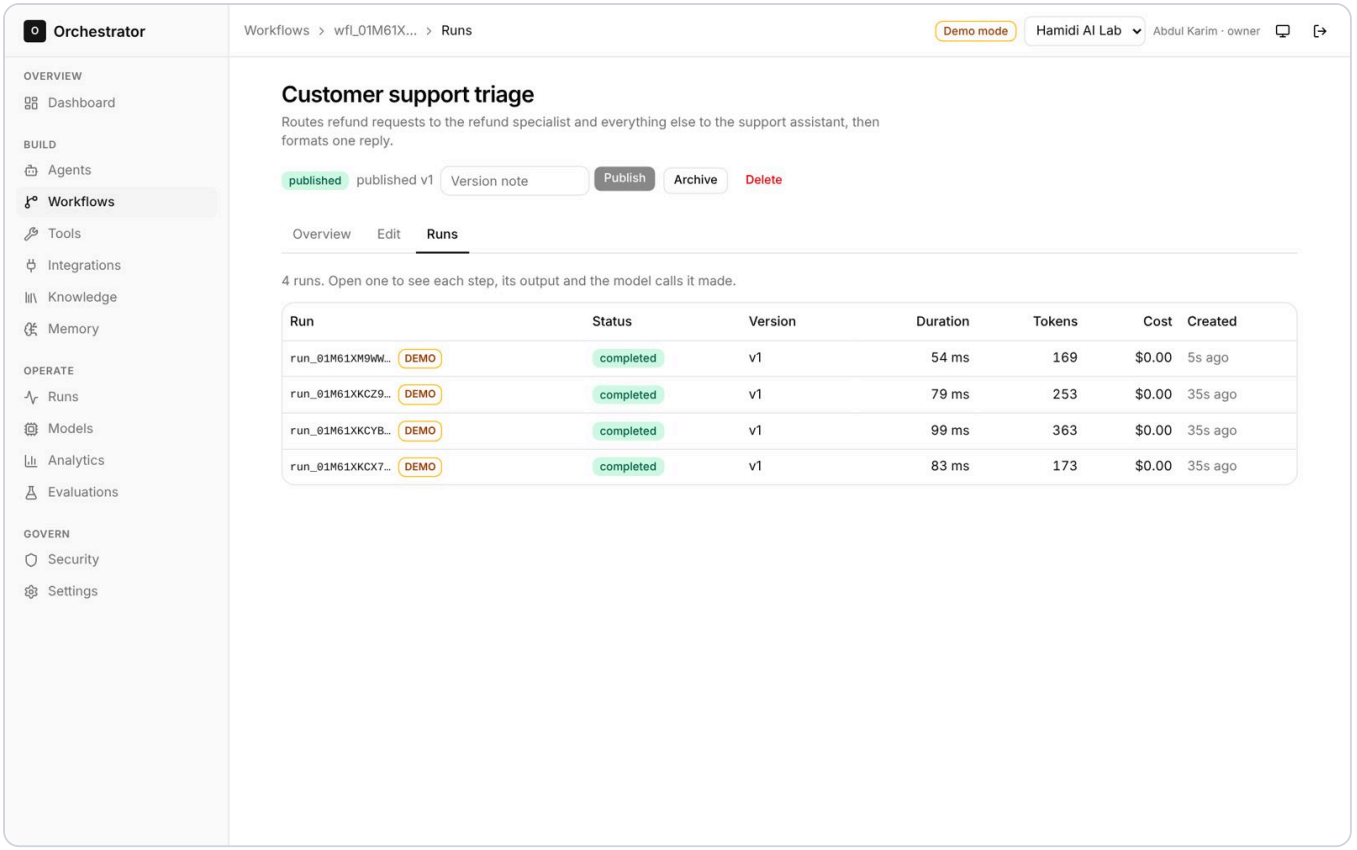
14. Edit workflow `/workflows/{id}/edit`

Edit the steps visually or as JSON. Templates such as {{ input.message }}, {{ steps.lookup.text }} and {{ last.text }} connect the steps. A loop guard limits how many steps a run may execute.



15. Workflow runs `/workflows/{id}/runs`

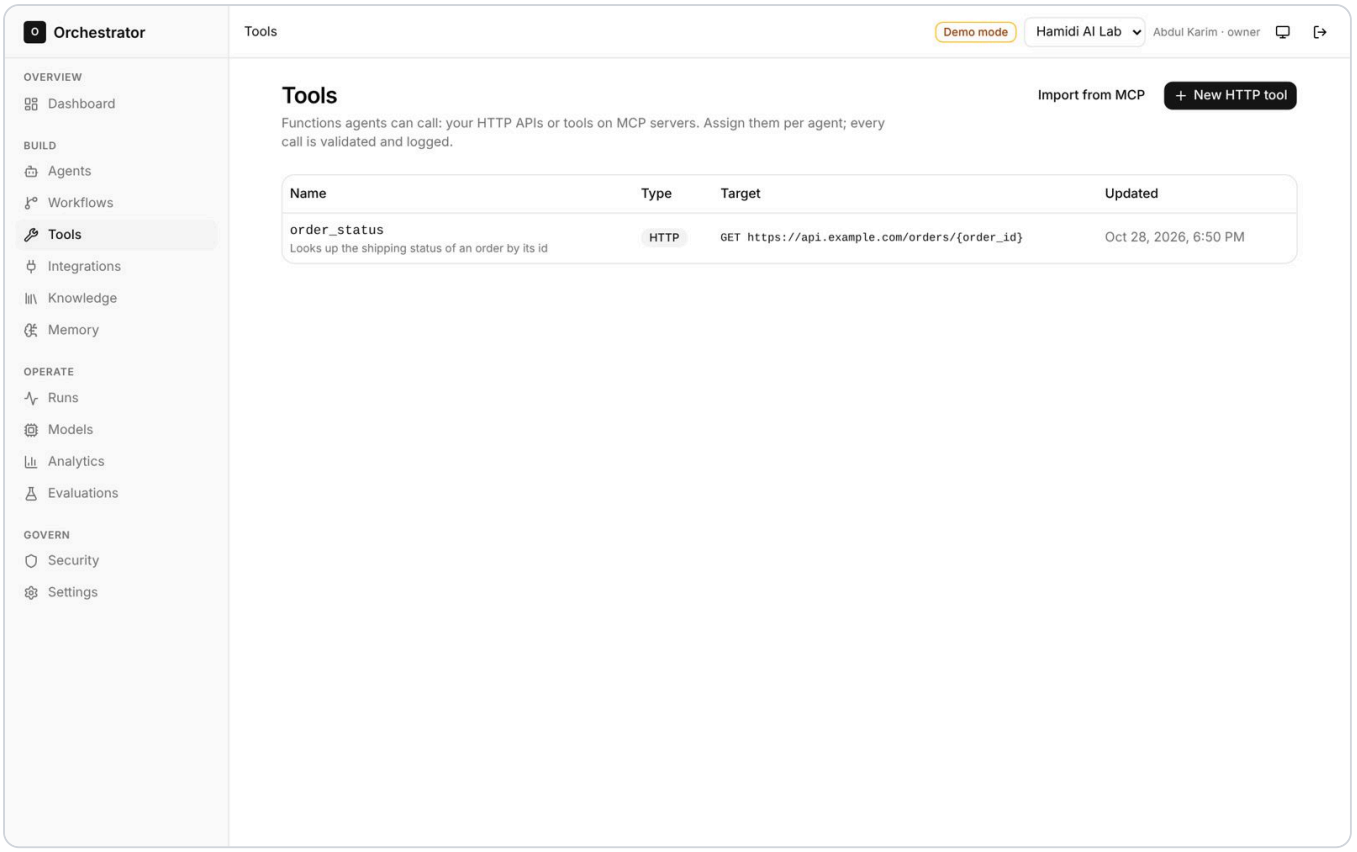
Every run of this workflow with status, version, duration, tokens and cost.



Build: tools, integrations, knowledge and memory

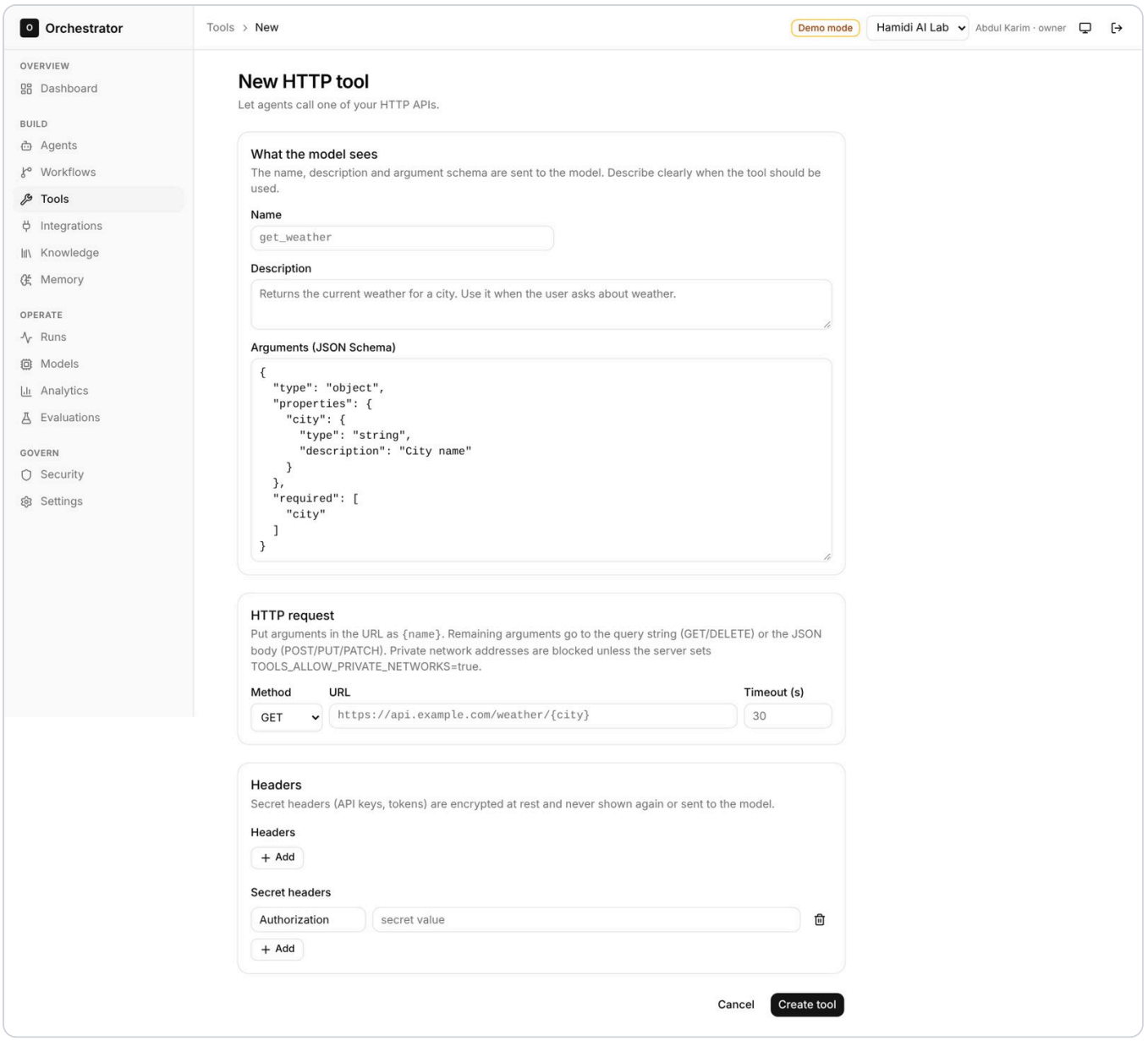
16. Tools `/tools`

HTTP API tools and tools imported from MCP servers. Agents may call only the tools on their allowlist.



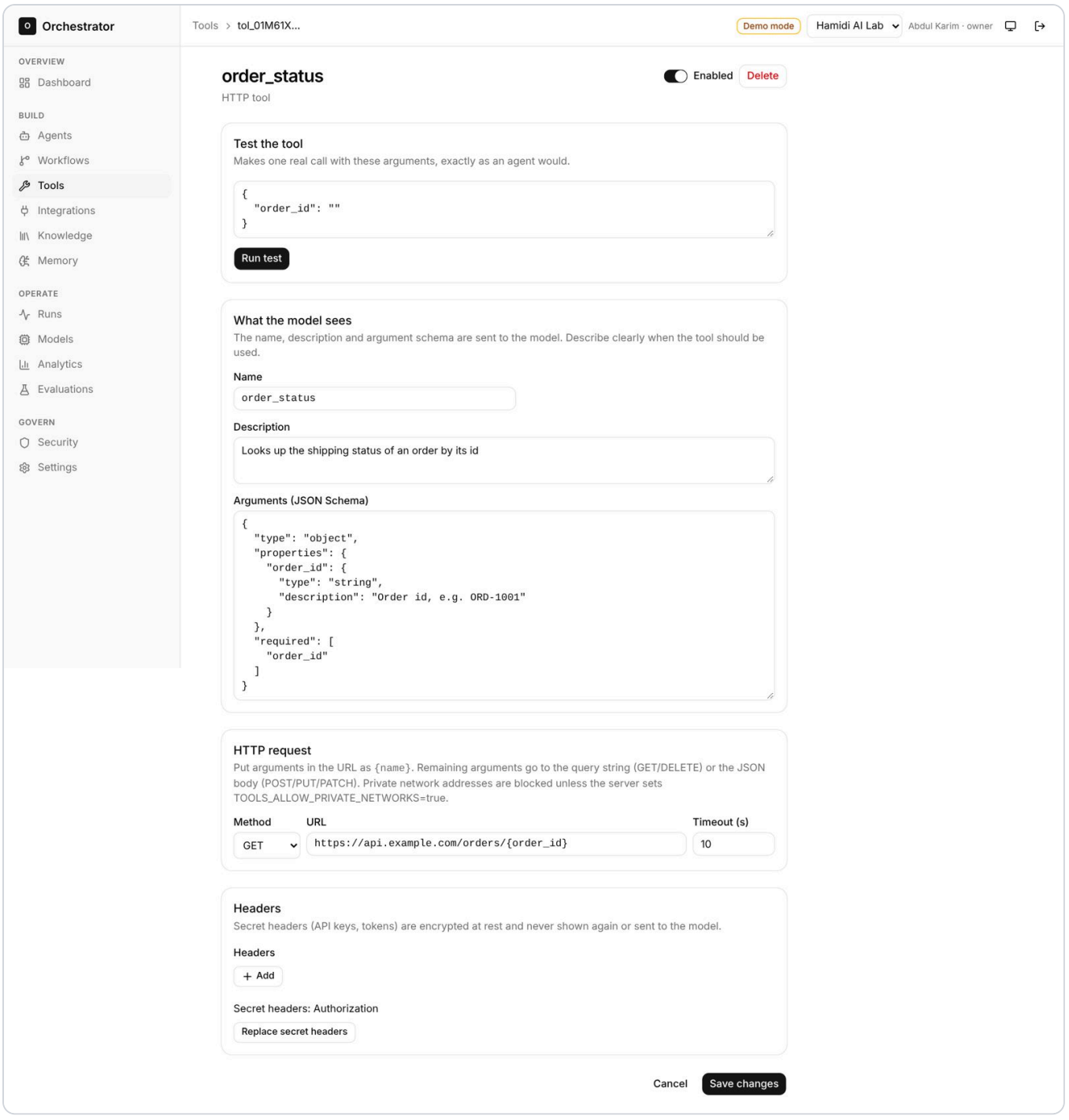
17. New tool `/tools/new`

Method, URL template, JSON Schema for the arguments, plain headers and encrypted secret headers (never sent back to the browser).



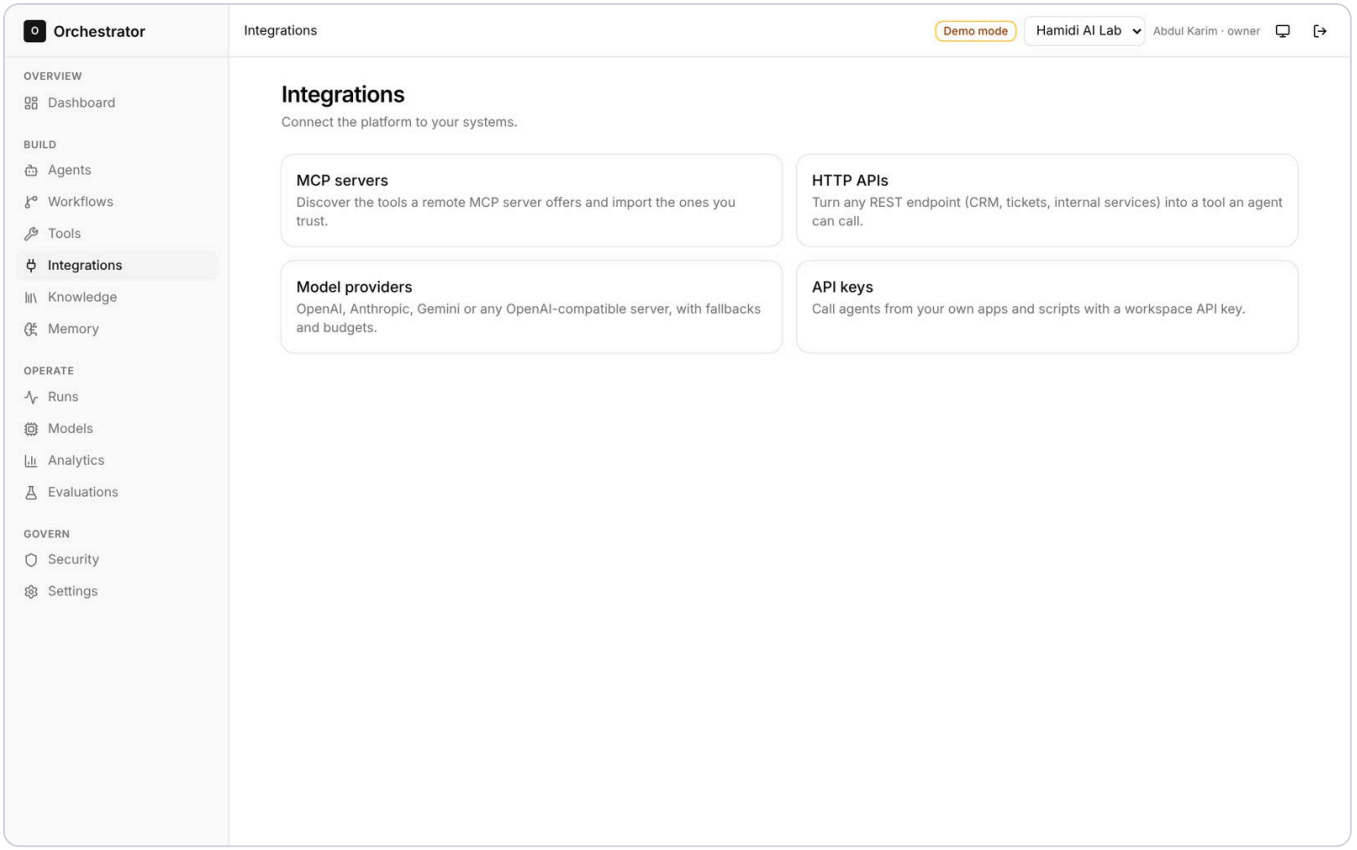
18. Tool detail and test console `/tools/{id}`

Edit a tool, enable or disable it, and test-call it with arguments. Calls are schema-validated, size-limited, time-limited and protected against SSRF.



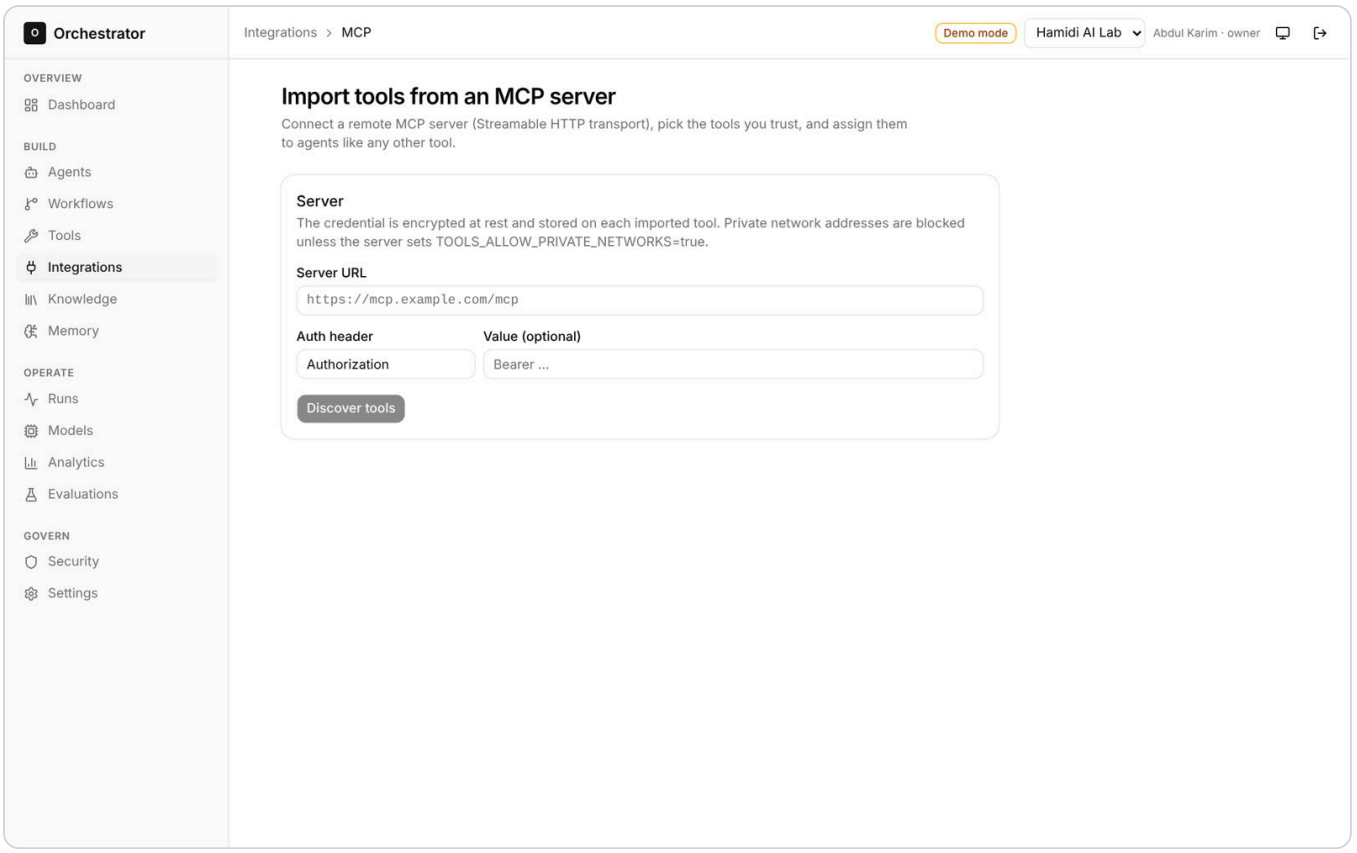
19. Integrations `/integrations`

Entry point for external connections: MCP servers and model providers.



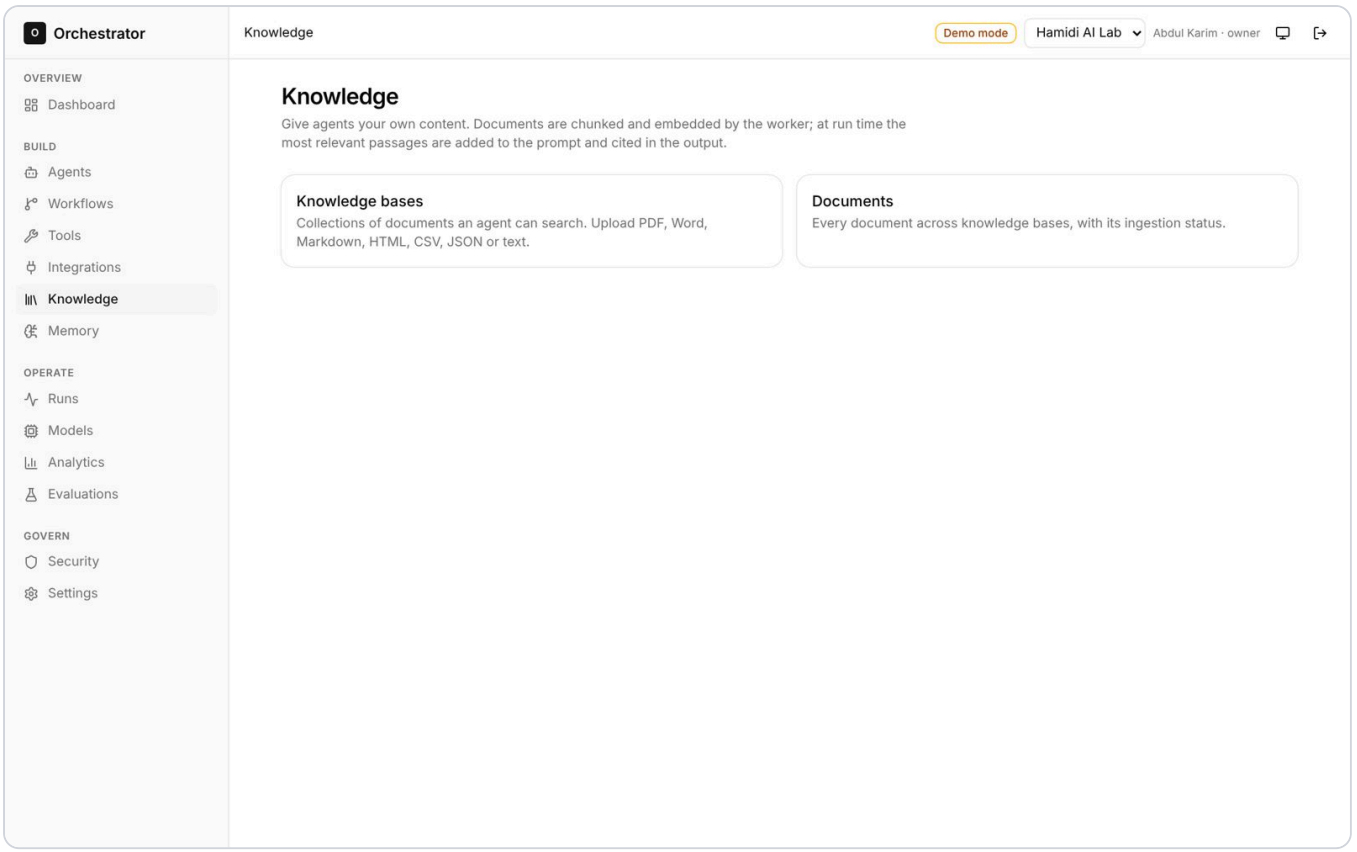
20. MCP import `/integrations/mcp`

Connect to a remote MCP server (Streamable HTTP), discover its tools and import the ones you choose as workspace tools.



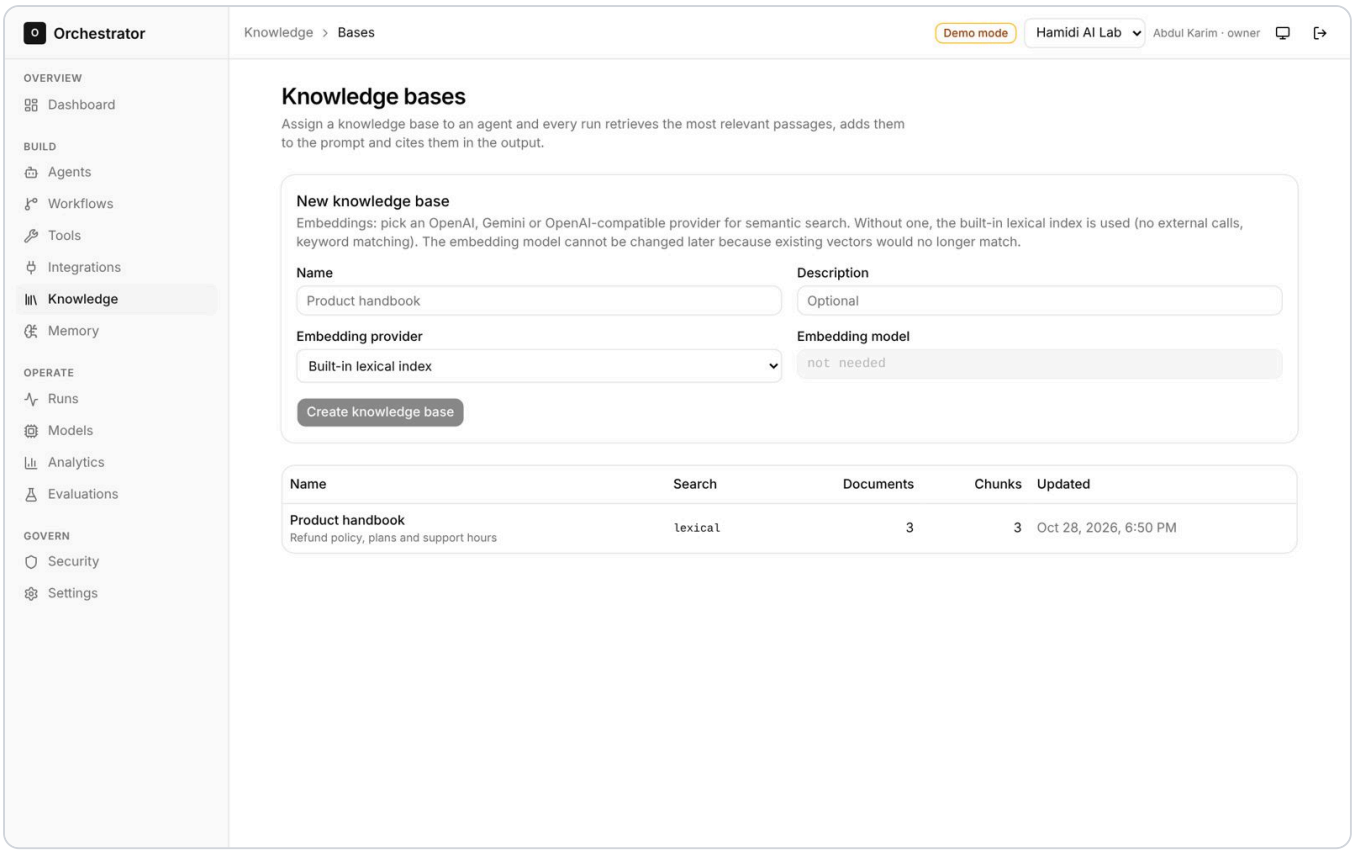
21. Knowledge `/knowledge`

Overview of retrieval-augmented generation (RAG): knowledge bases and documents.



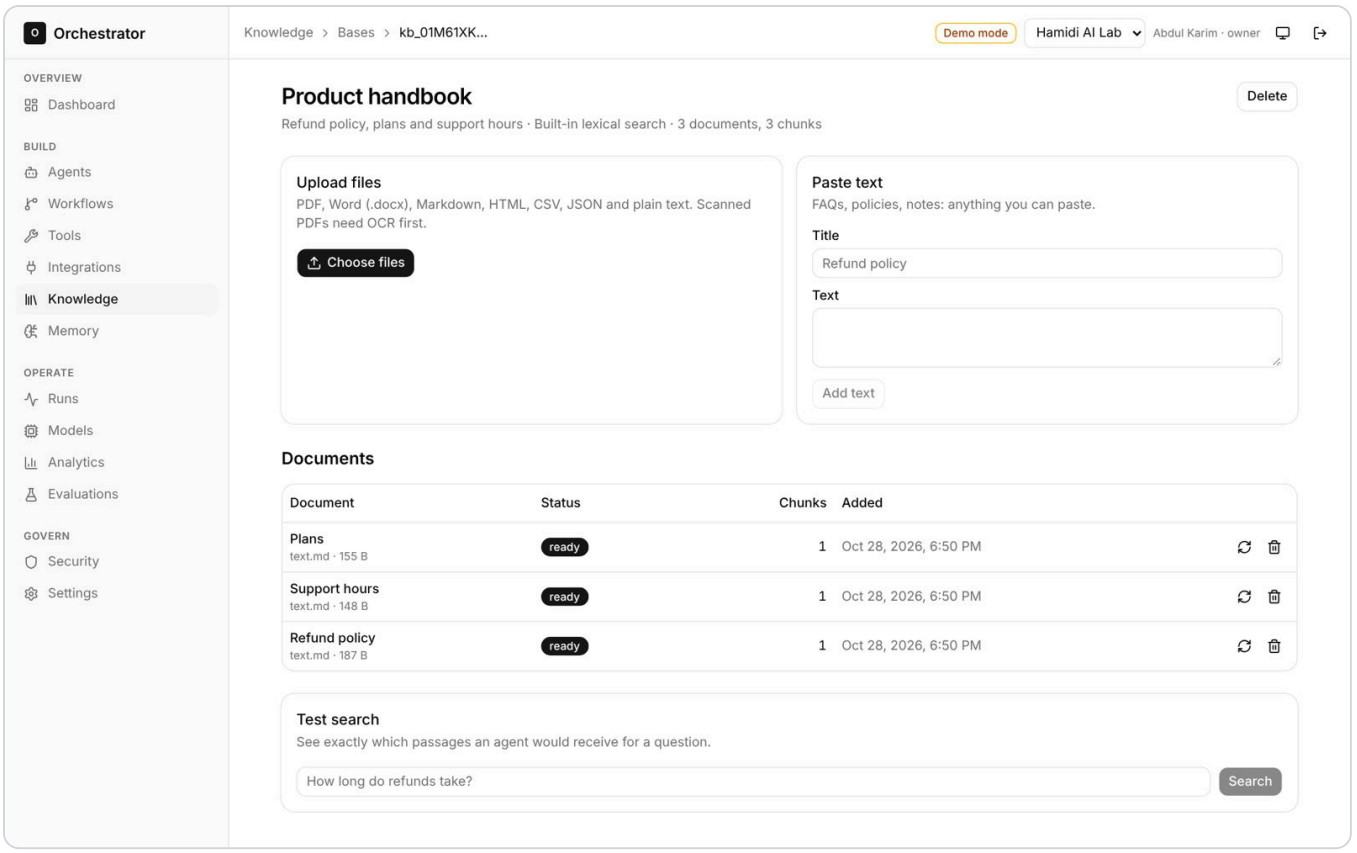
22. Knowledge bases `/knowledge/bases`

Create knowledge bases that use OpenAI, Gemini or OpenAI-compatible embeddings, or the built-in lexical index, which needs no external calls.



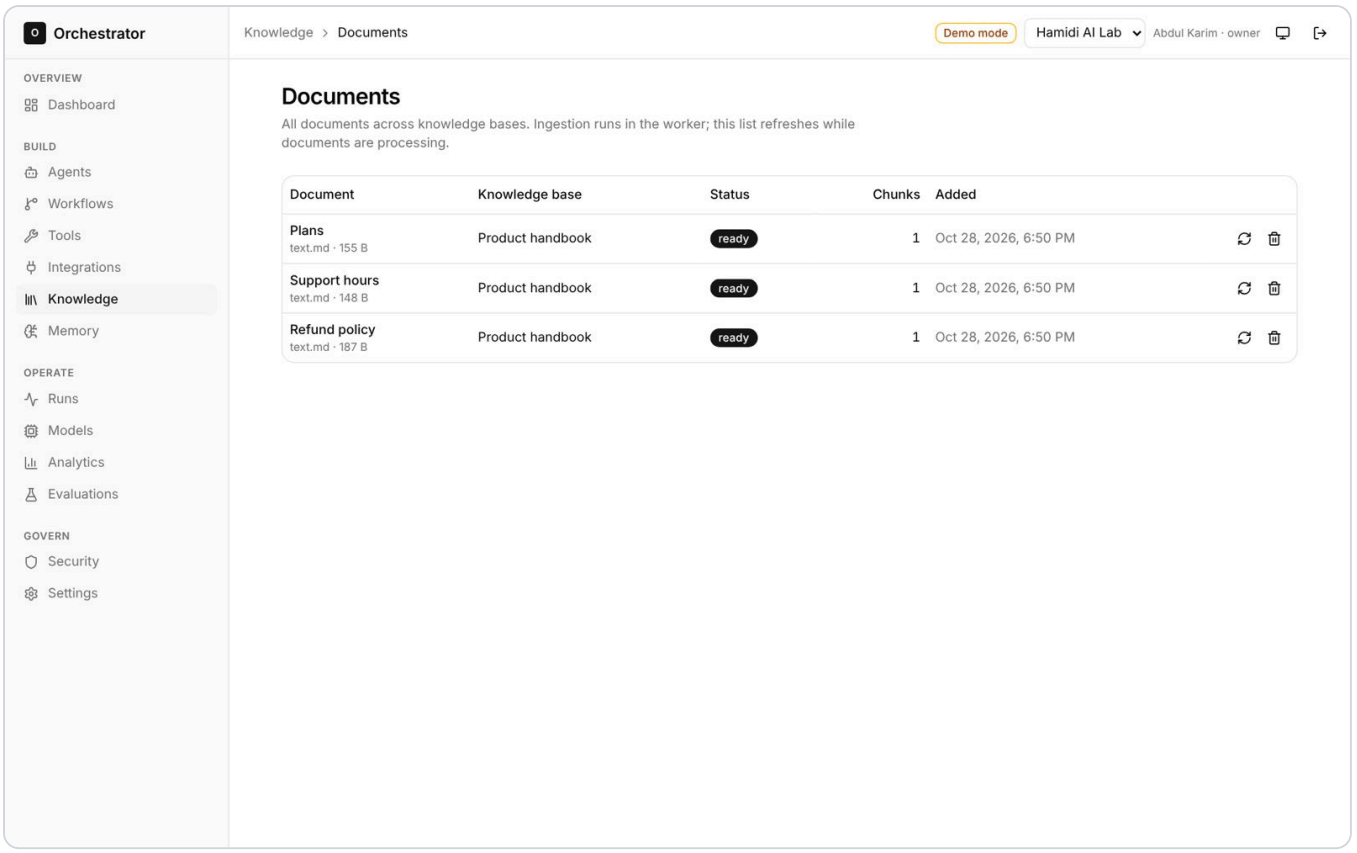
23. Knowledge base detail `/knowledge/bases/{id}`

Upload PDF, DOCX, Markdown, HTML, CSV, JSON or text files, or paste text. The worker extracts, chunks and indexes them in the background. A search box tests retrieval.



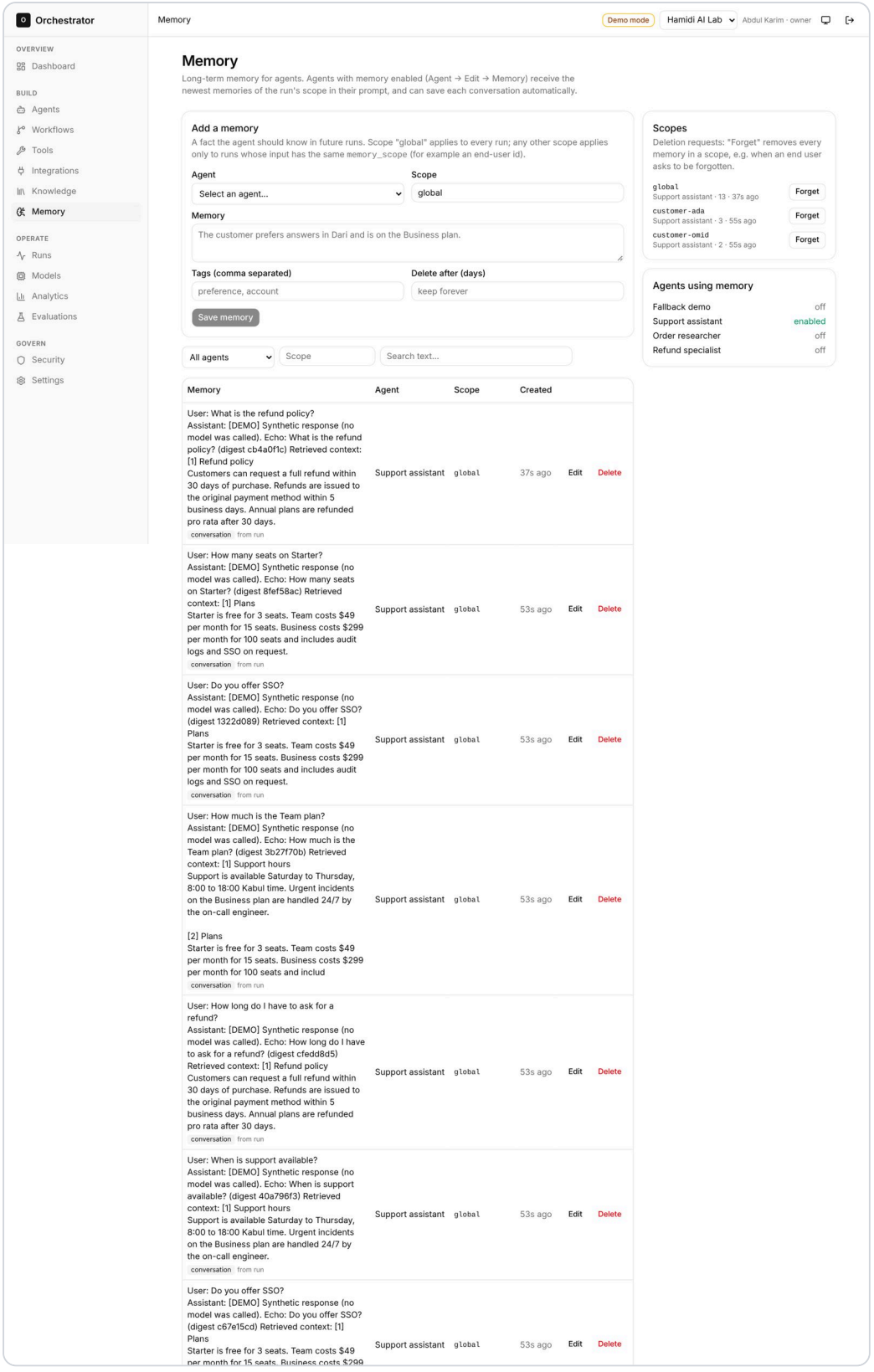
24. Documents `/knowledge/documents`

Every document across knowledge bases with its ingestion status, size and chunk count, plus reindex and delete actions.



25. Memory (new) `/memory`

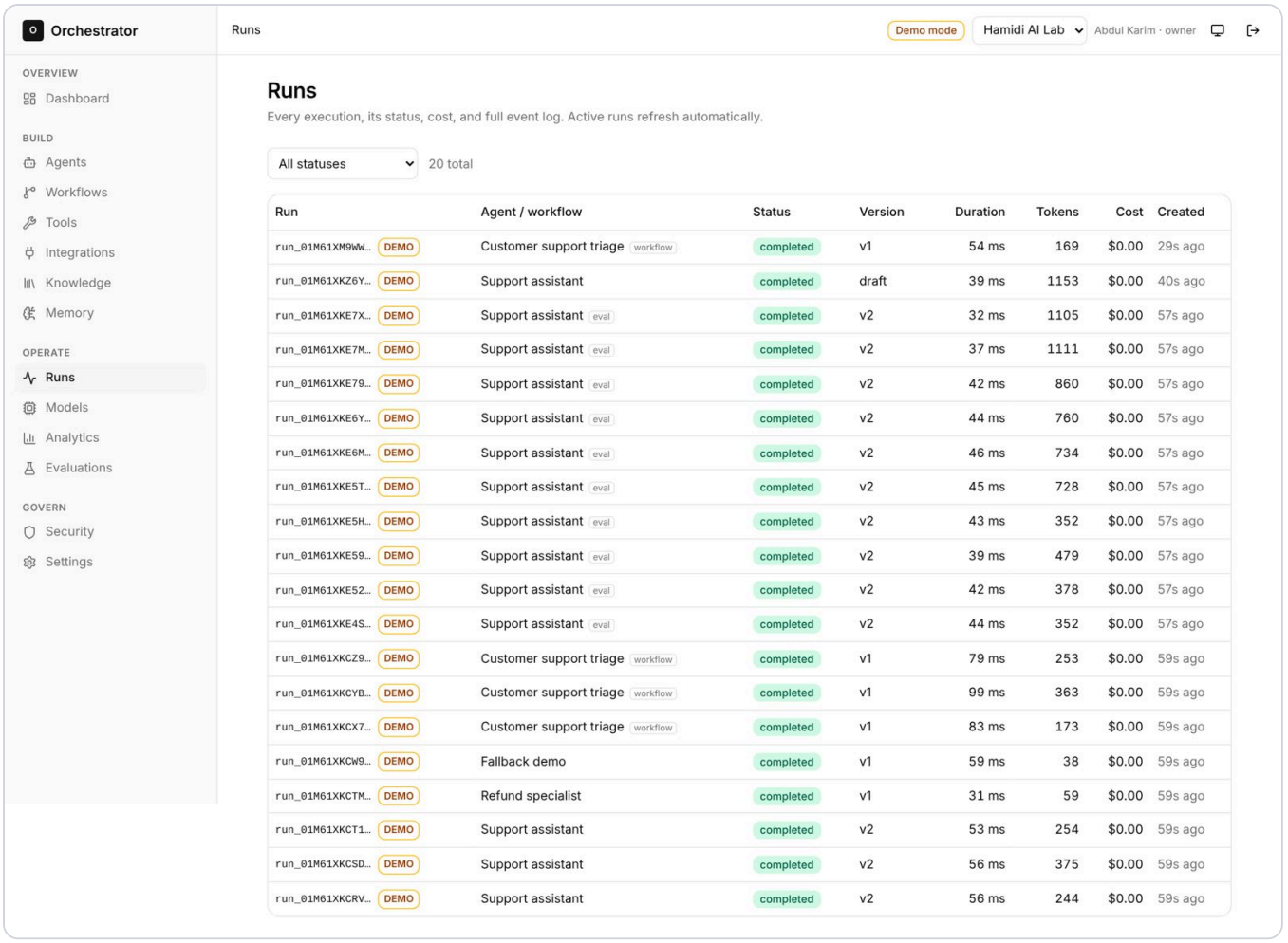
Long-term agent memory. A memory has a scope: "global" applies to every run, and any other scope, such as an end-user id passed as input.memory_scope, applies only to that user's runs. Agents with memory enabled get the newest memories in their prompt and can save each conversation automatically. You can add, edit, search, set an expiry, and "Forget" a whole scope when a user asks to be deleted. (The screenshot is cut off: it lists the saved conversations.)



Operate

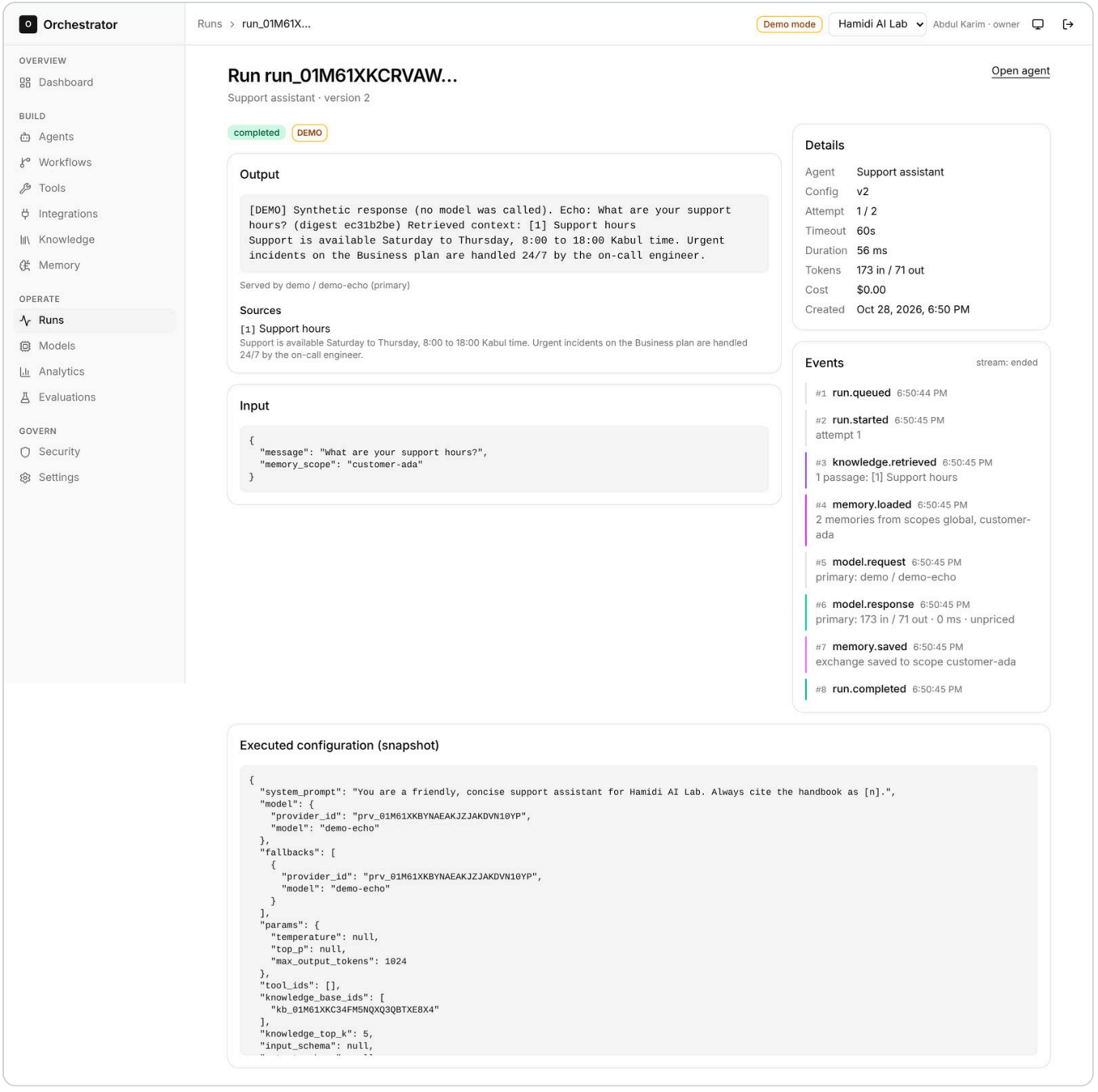
26. Runs `/runs`

Every agent, workflow and evaluation run with status, version, duration, tokens and cost. Runs are queued in MongoDB and executed by the worker, with leases, crash recovery, timeouts and cancellation.



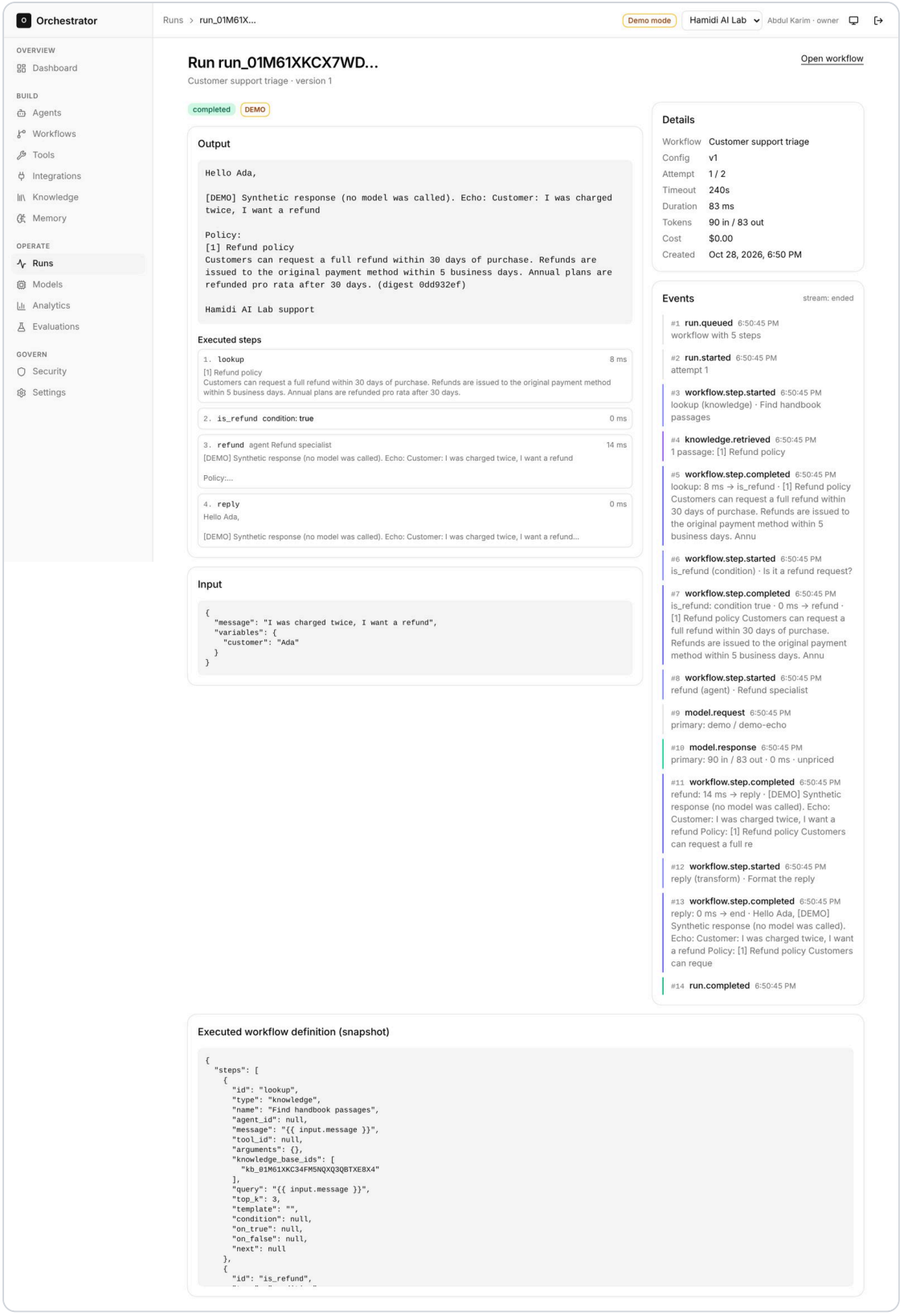
27. Run inspector (agent) `/runs/{id}`

Output, citations, input, details and the full event log. This run loaded the customer's memories, retrieved handbook passages, called the model and saved the exchange as a new memory.



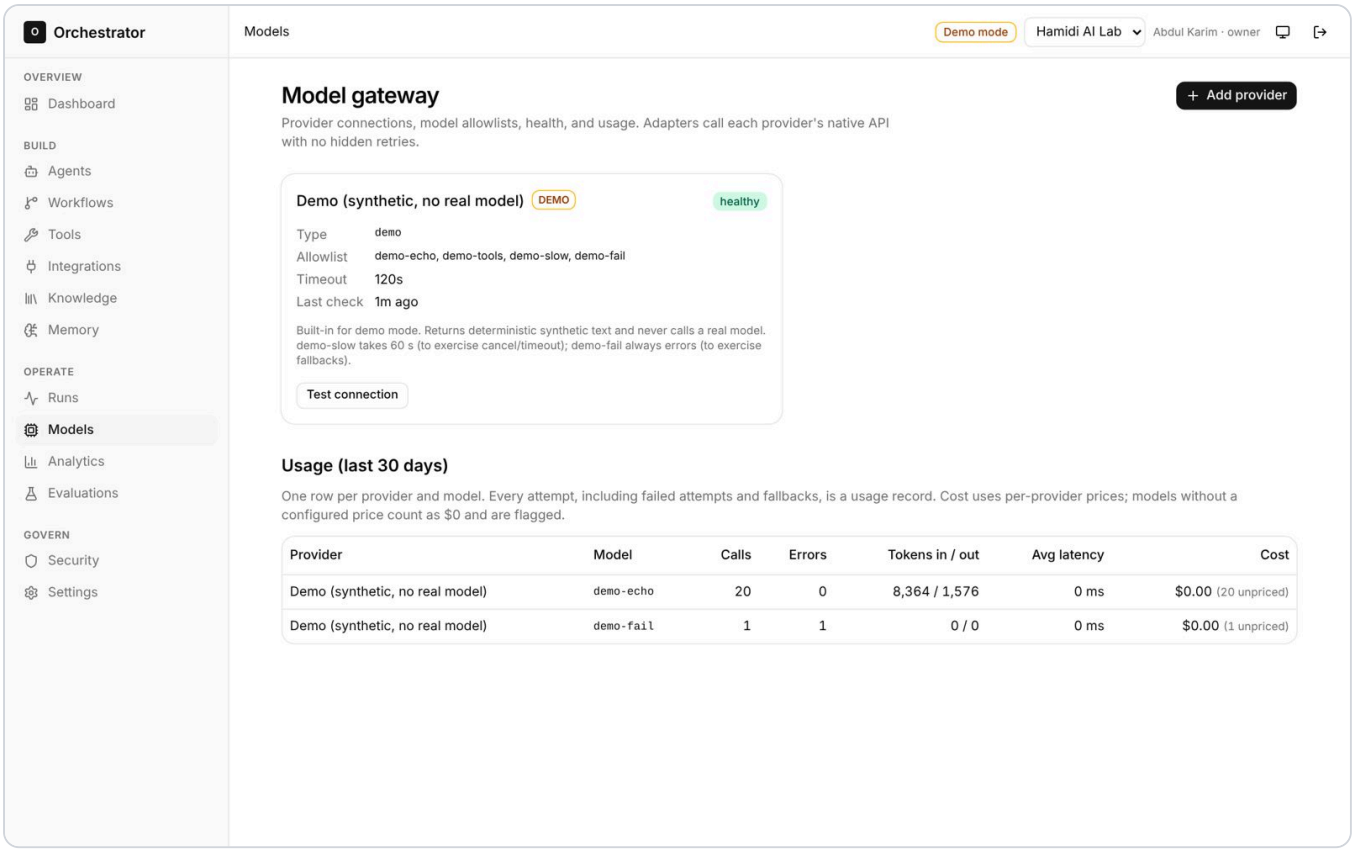
28. Run inspector (workflow) `/runs/{id}`

For workflow runs the inspector also lists the executed steps in order, with each step's output and timing.



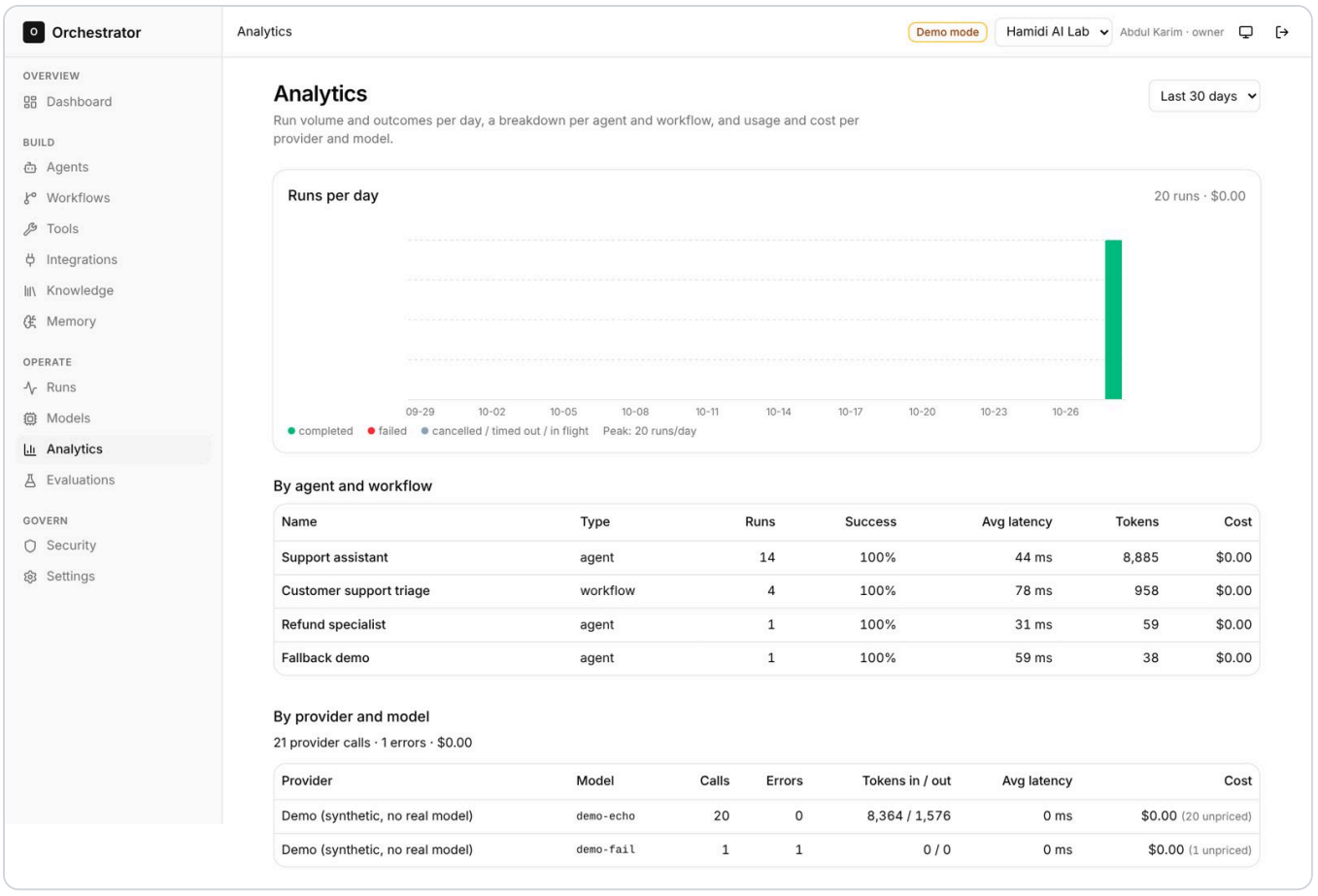
29. Models `/models`

Model providers: OpenAI, Anthropic, Gemini, OpenAI-compatible endpoints and the built-in Demo provider. Keys are encrypted at rest. Each provider has a connection test, health status, model allowlist and price table.



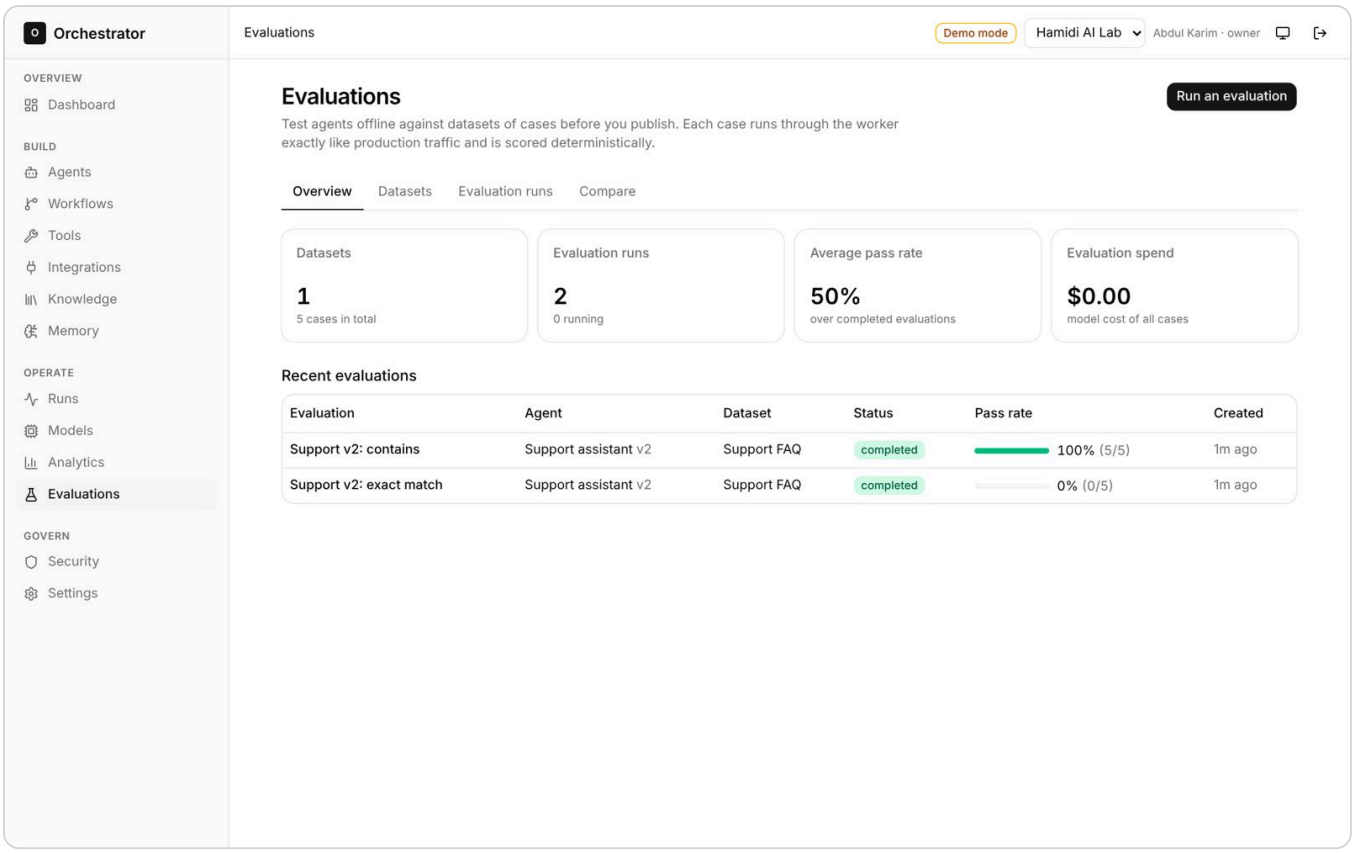
30. Analytics (completed) `/analytics`

Runs per day by outcome, a breakdown per agent and per workflow (runs, success rate, latency, tokens, cost) and usage per provider and model.



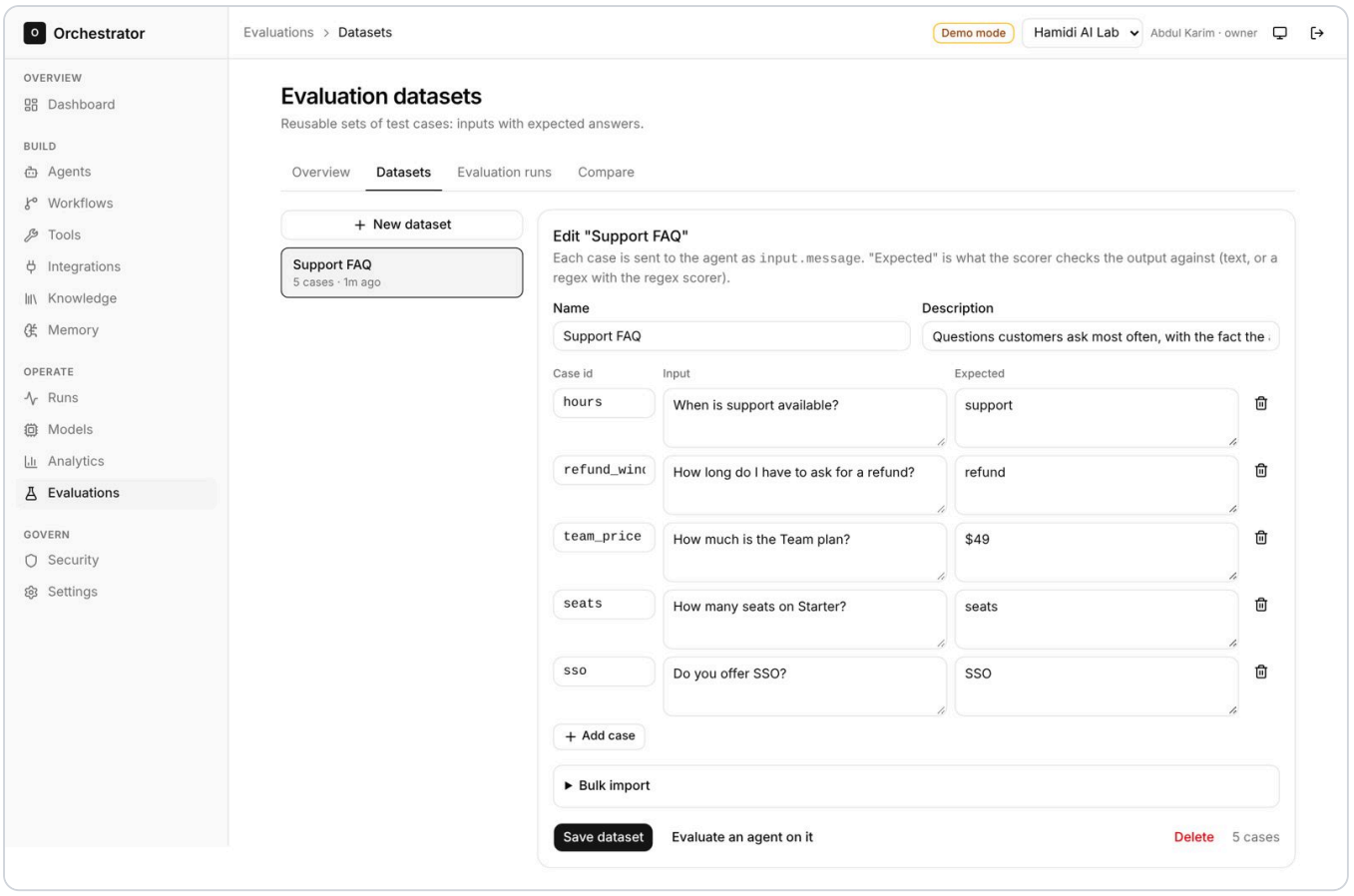
31. Evaluations (new) `/evaluations`

Overview of offline testing: datasets, evaluation runs, average pass rate, spend and recent evaluations.



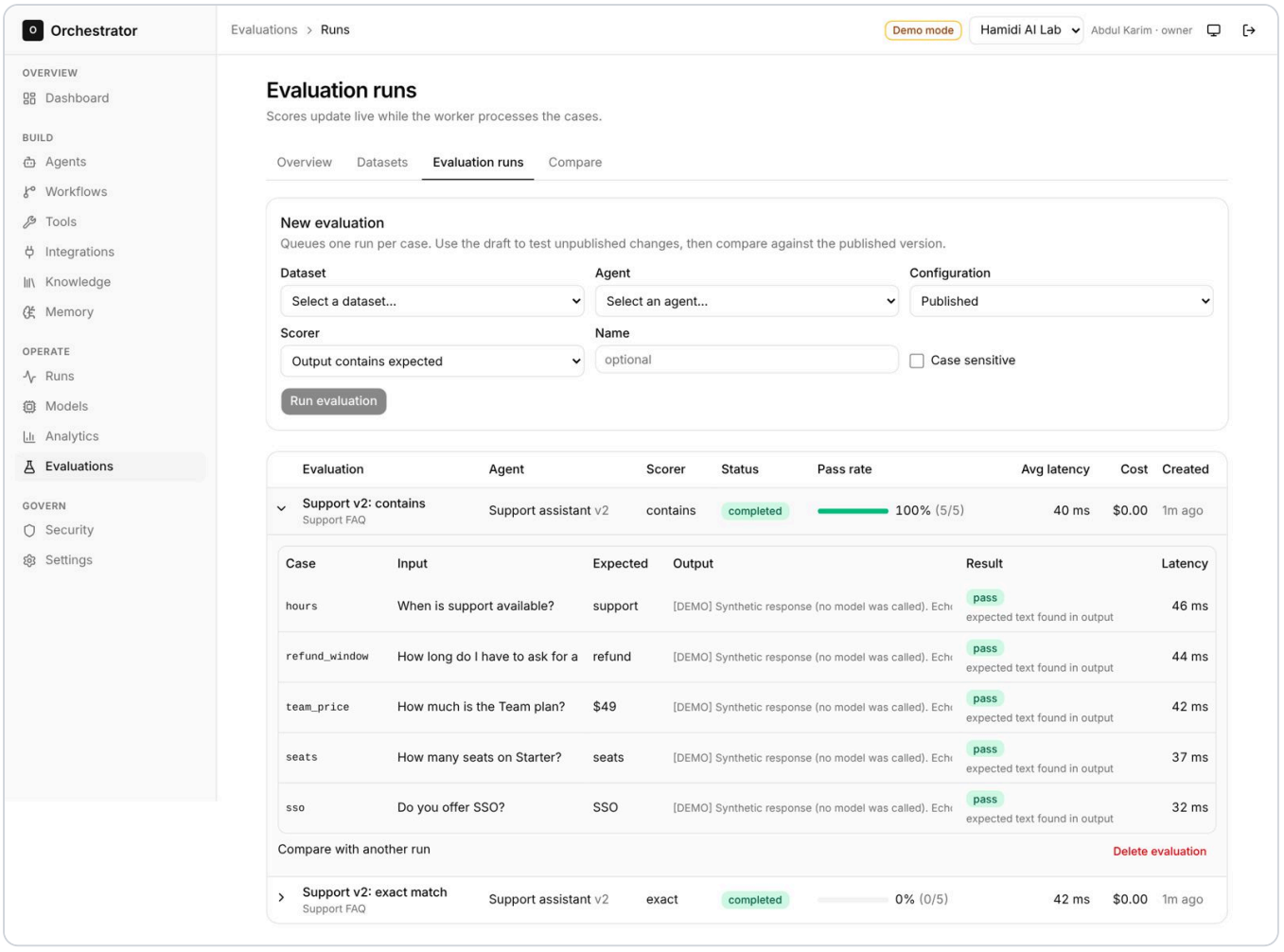
32. Evaluation datasets `/evaluations/datasets`

Datasets of test cases (an input plus the expected answer), edited in a table or bulk-imported from JSON, JSON Lines or "input | expected" lines.



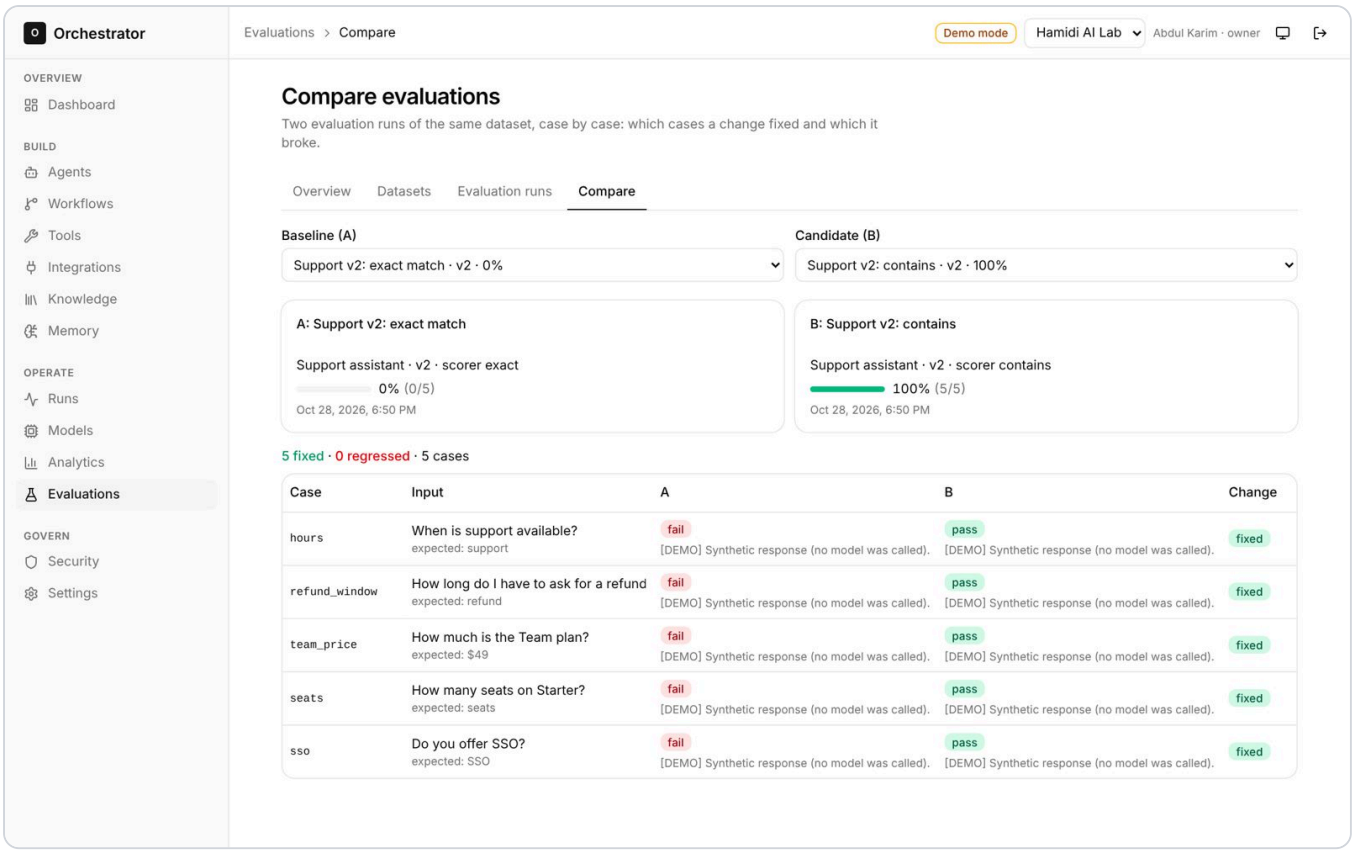
33. Evaluation runs `/evaluations/runs`

Start an evaluation of an agent's published or draft configuration against a dataset with a scorer: contains, exact, not contains, regex, valid JSON or not empty. Each case runs through the worker like production traffic, and the pass rate updates live. Expand a run to see each case.



34. Compare evaluations `/evaluations/compare`

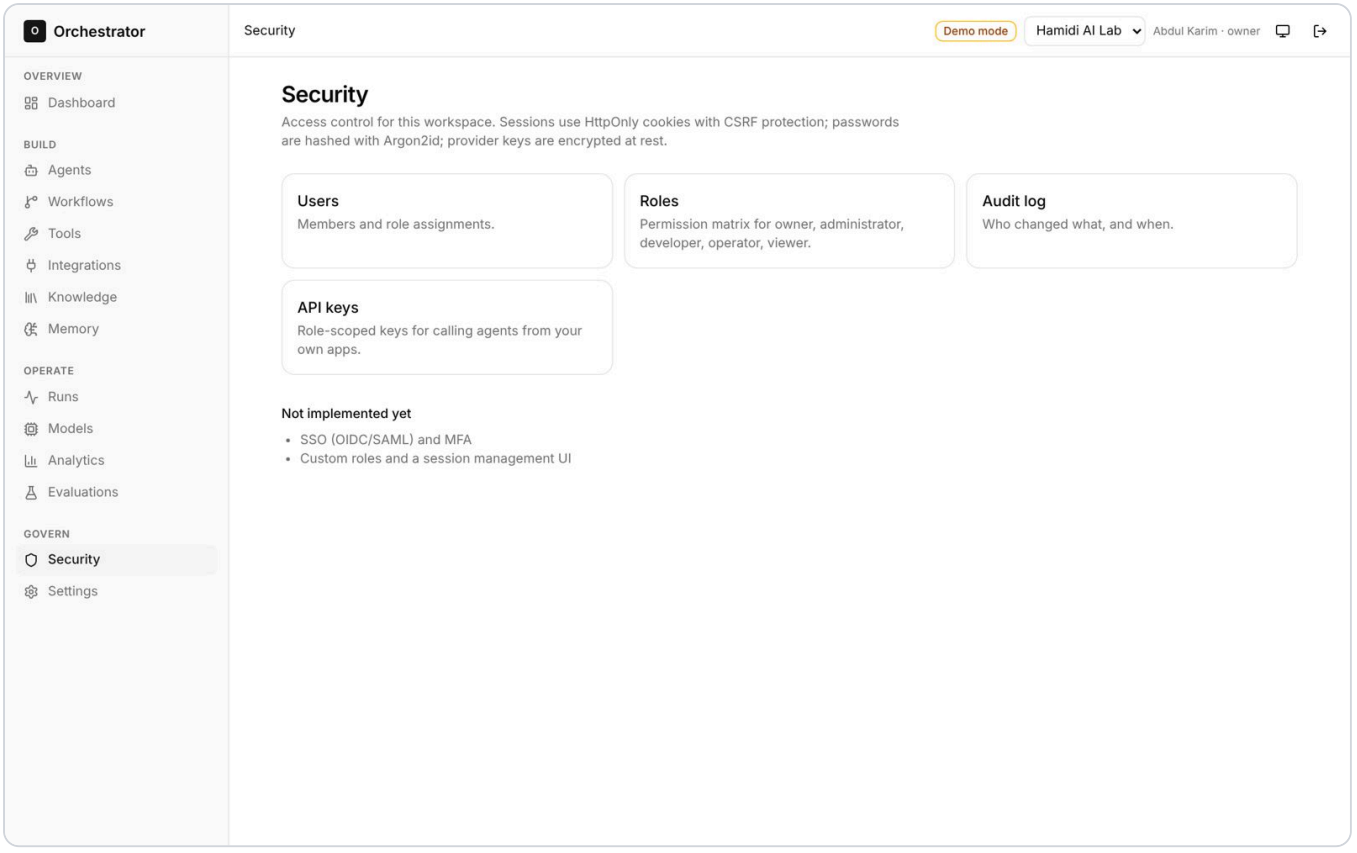
Two evaluation runs of the same dataset side by side, showing which cases were fixed and which regressed. Use it to check a change before you publish it.



Govern

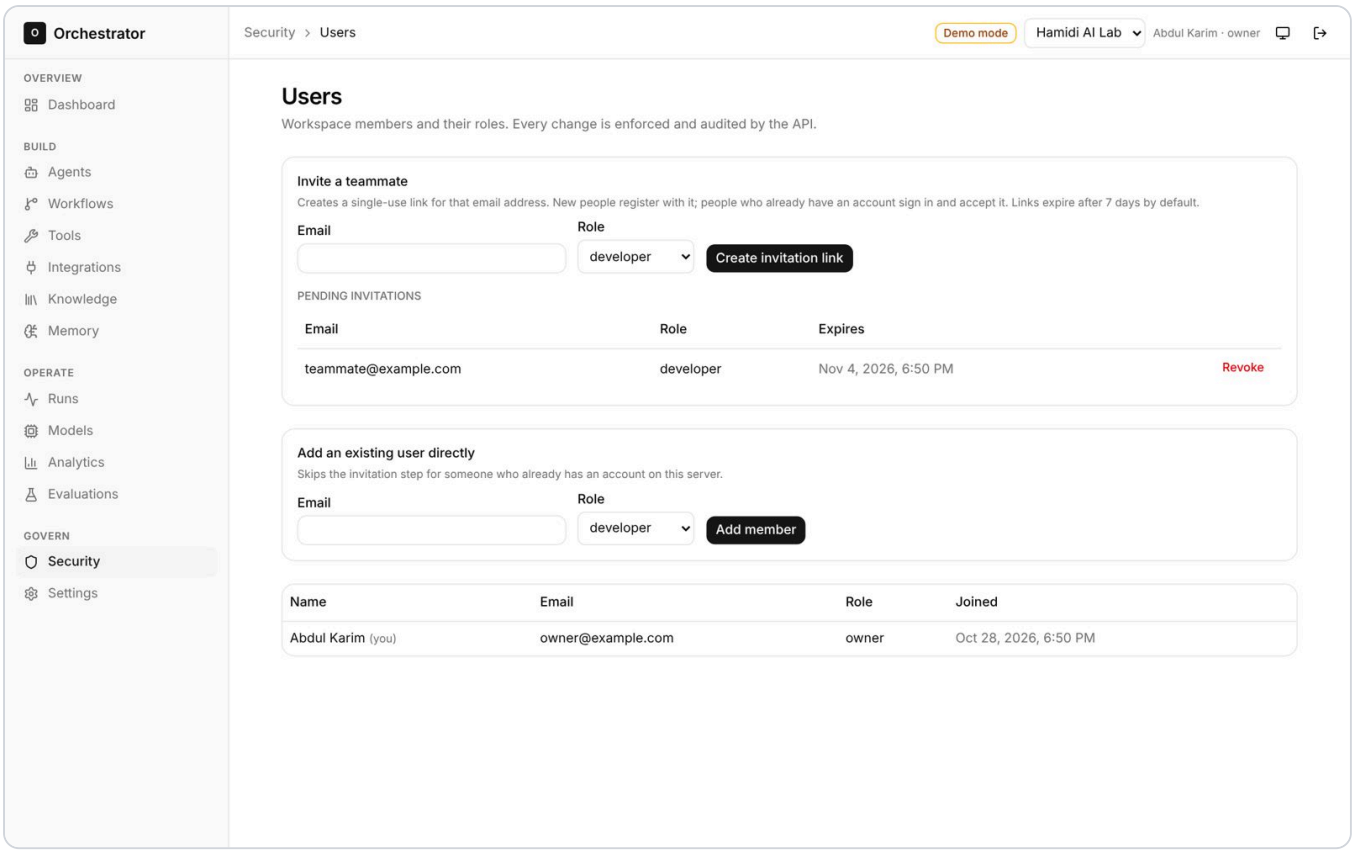
35. Security `/security`

Access control overview: users, roles, audit log and API keys.



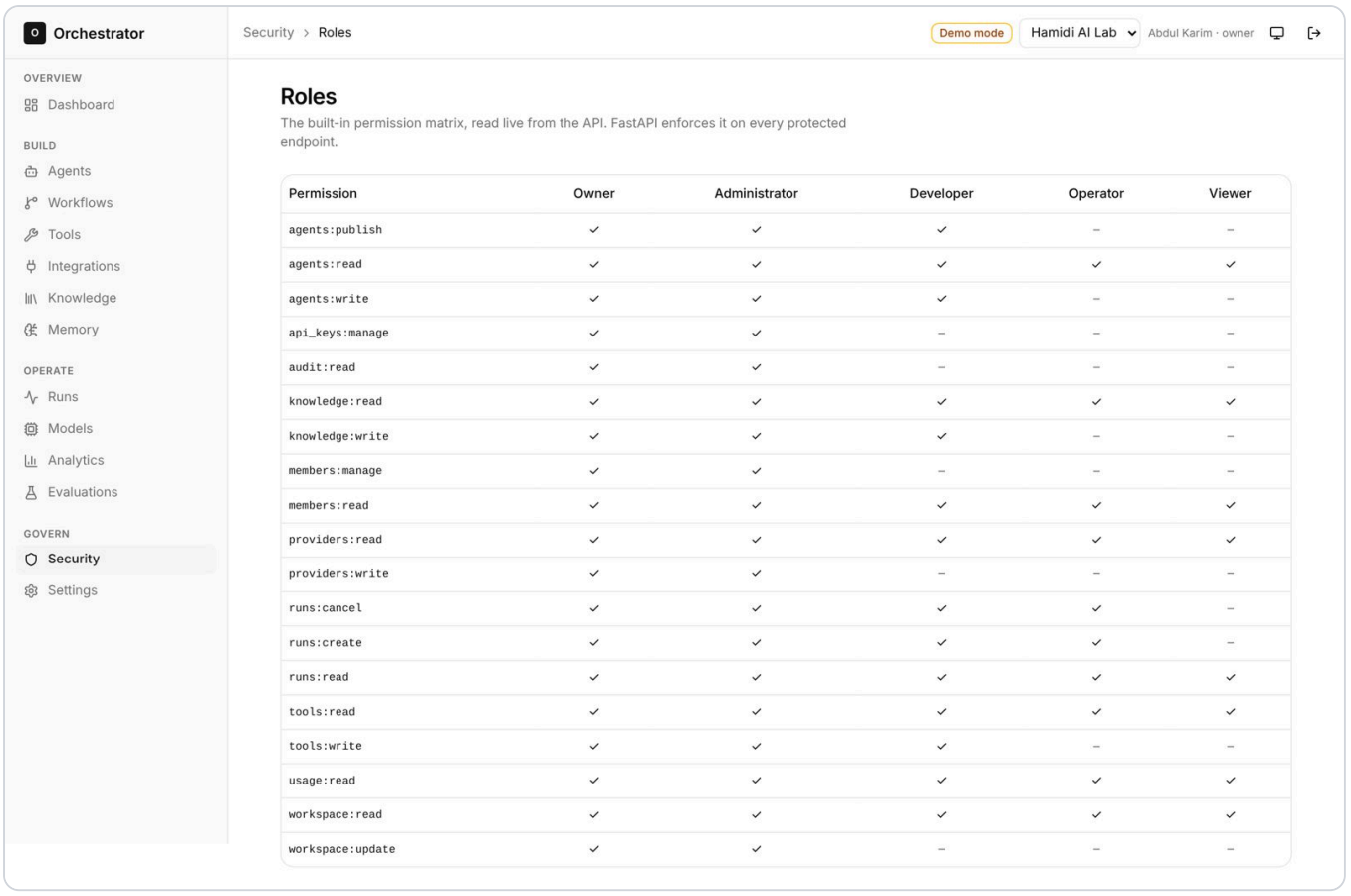
36. Users `/security/users`

Members and their roles, role changes and removal (with rules such as "a workspace keeps at least one owner"), and invitation links.



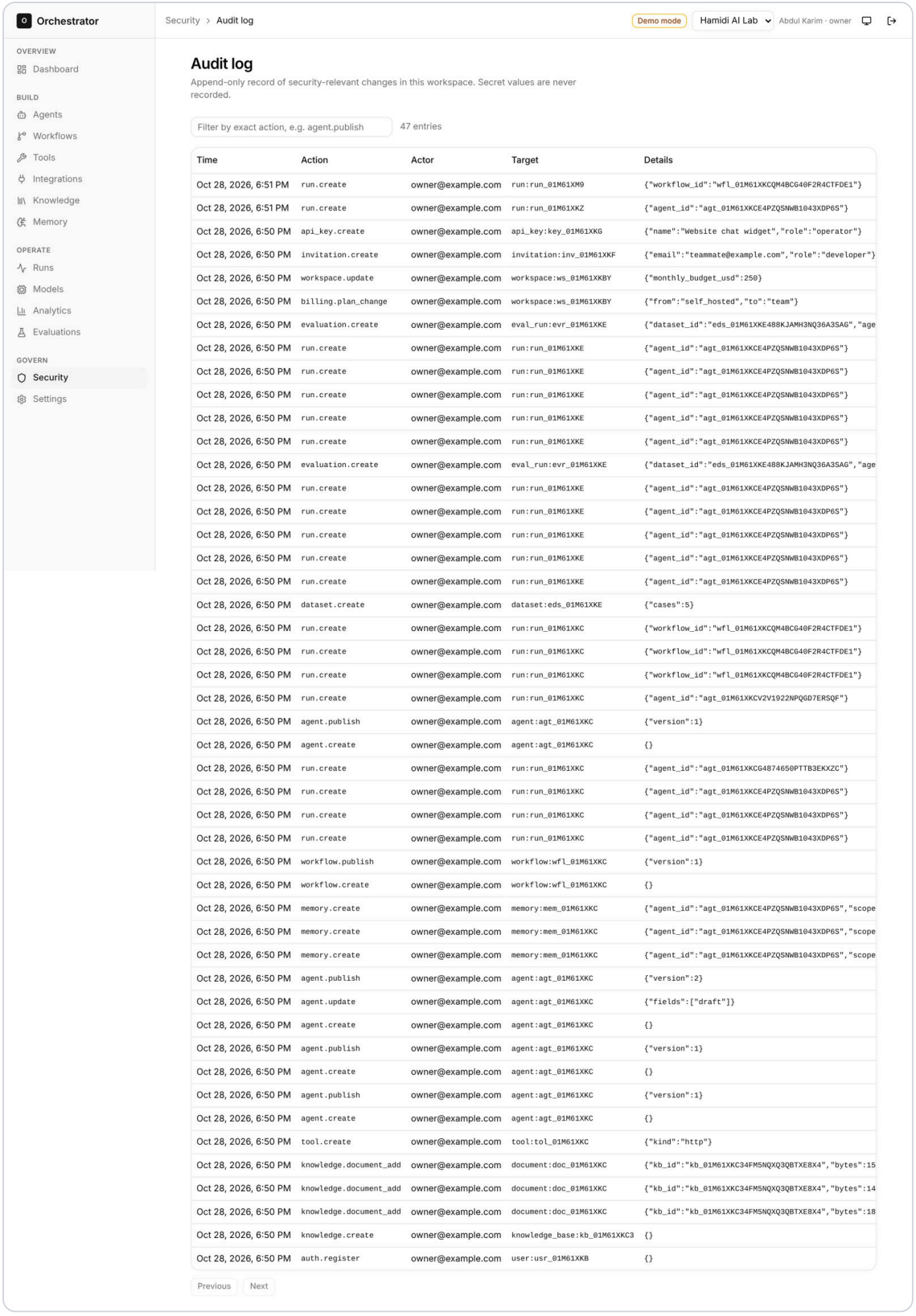
37. Roles `/security/roles`

The permission matrix for owner, administrator, developer, operator and viewer, read live from the API. FastAPI enforces it on every endpoint.



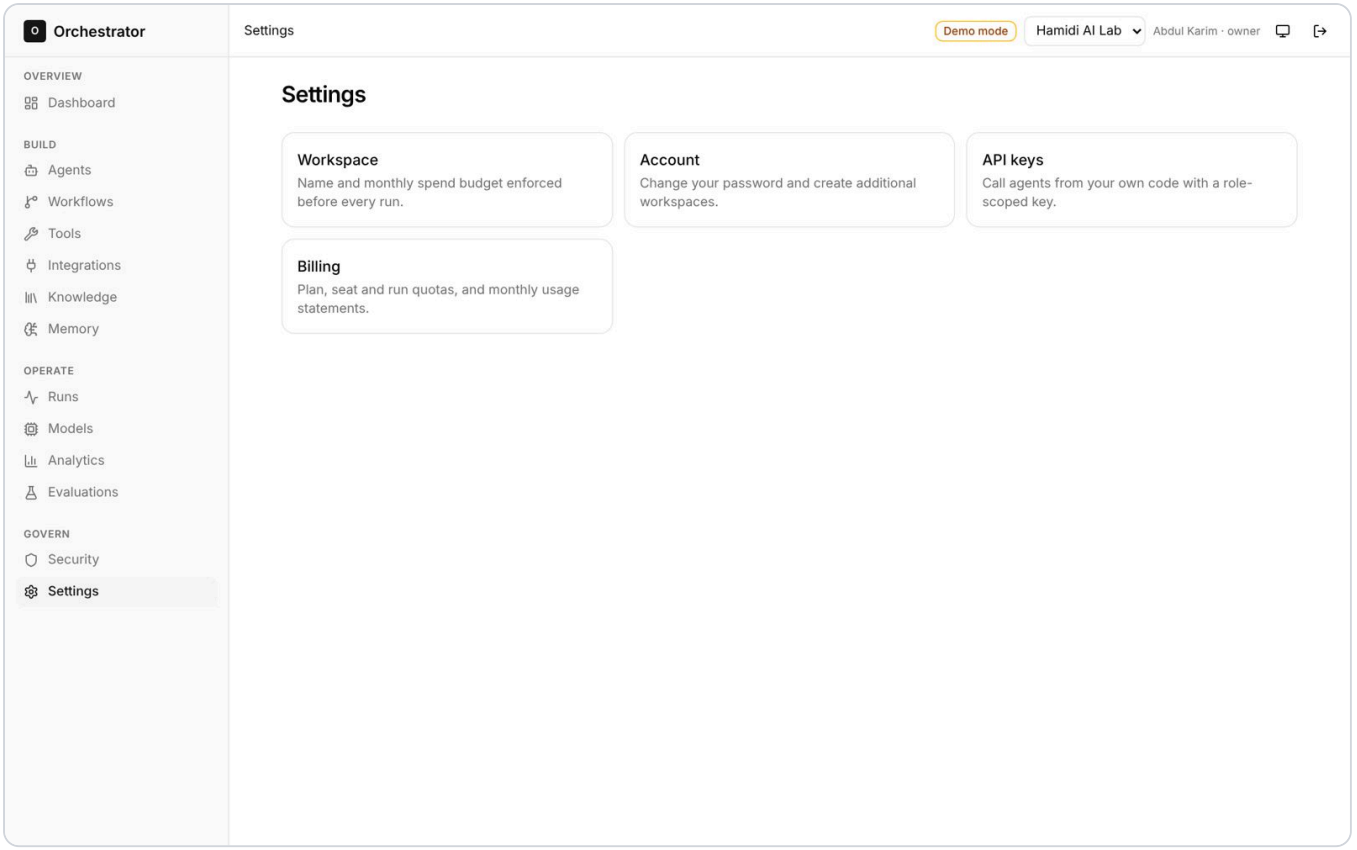
38. Audit log `/security/audit`

Who changed what and when: agents, workflows, runs, memories, datasets, plan changes, members and keys.



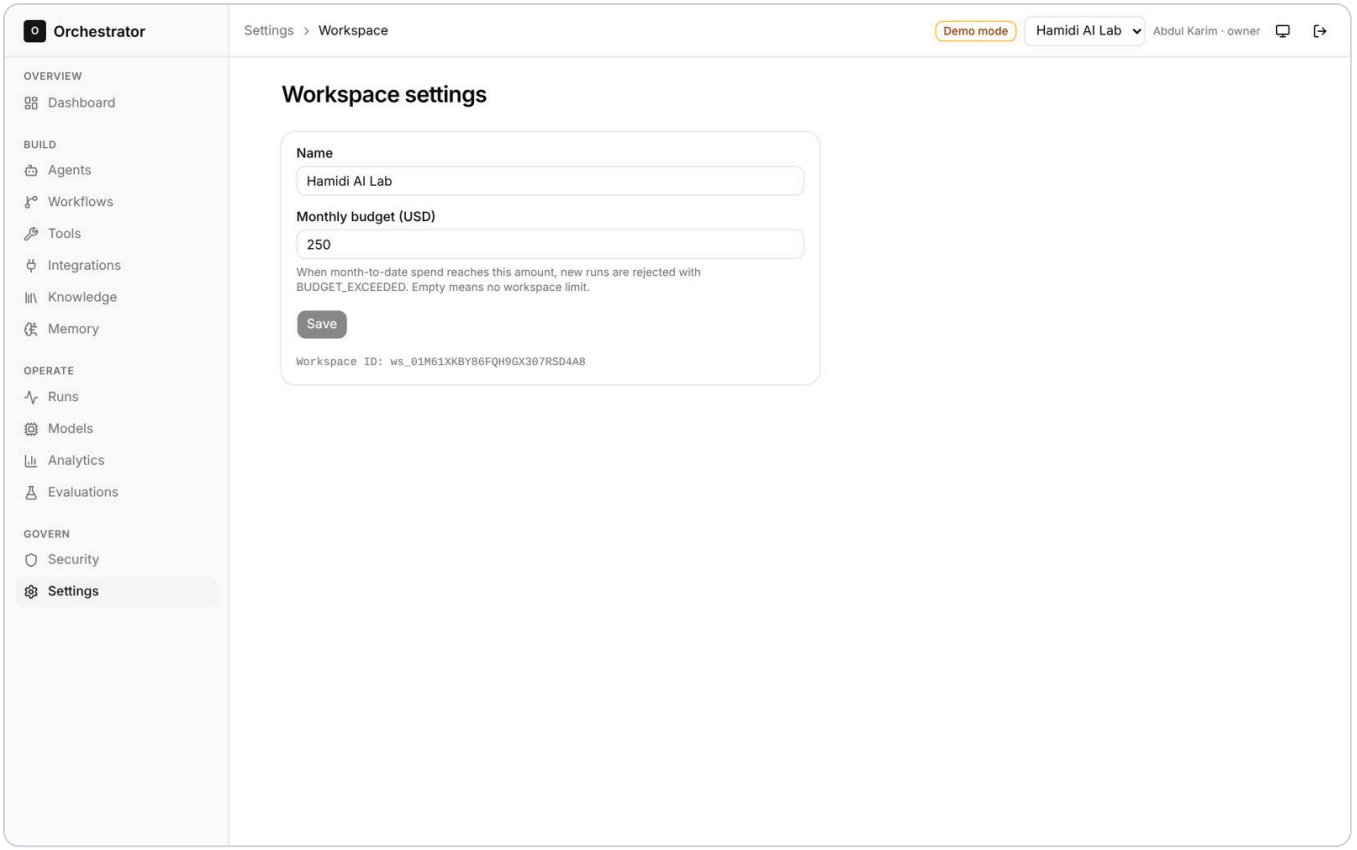
39. Settings `/settings`

Workspace, account, API keys and billing.



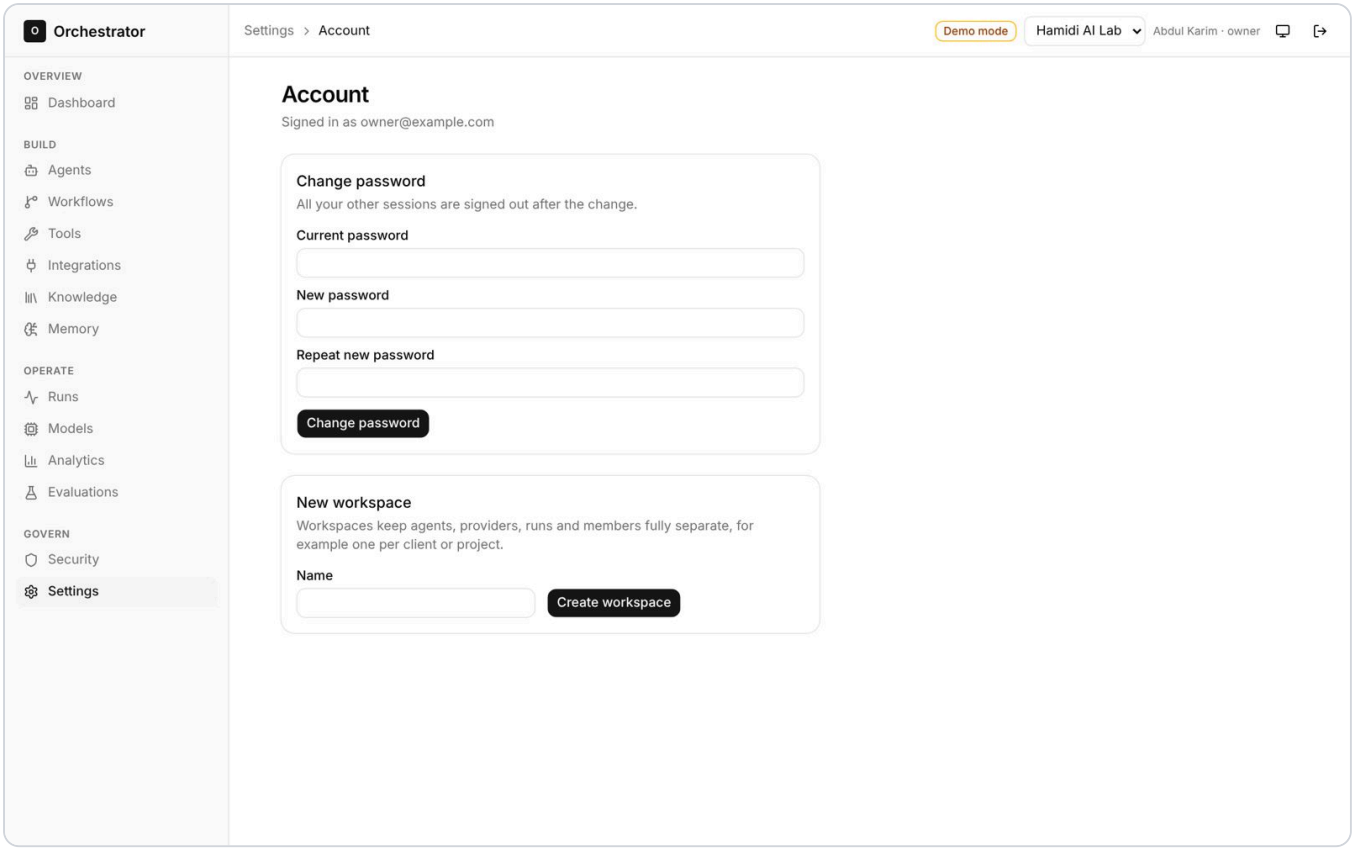
40. Workspace settings `/settings/workspace`

Workspace name and the monthly spend budget, which is checked before every model call.



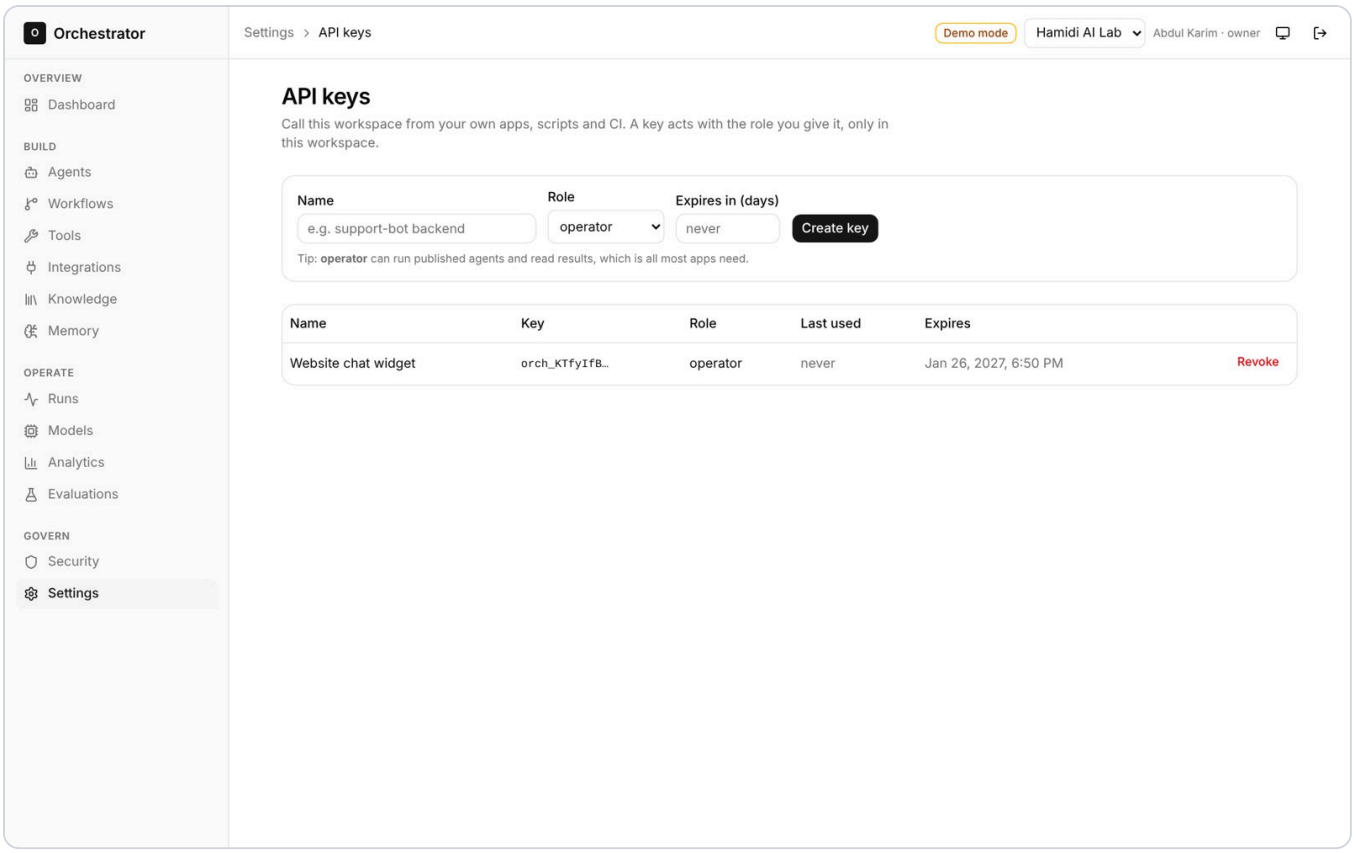
41. Account `/settings/account`

Change your password (this signs out other sessions) and create additional workspaces.



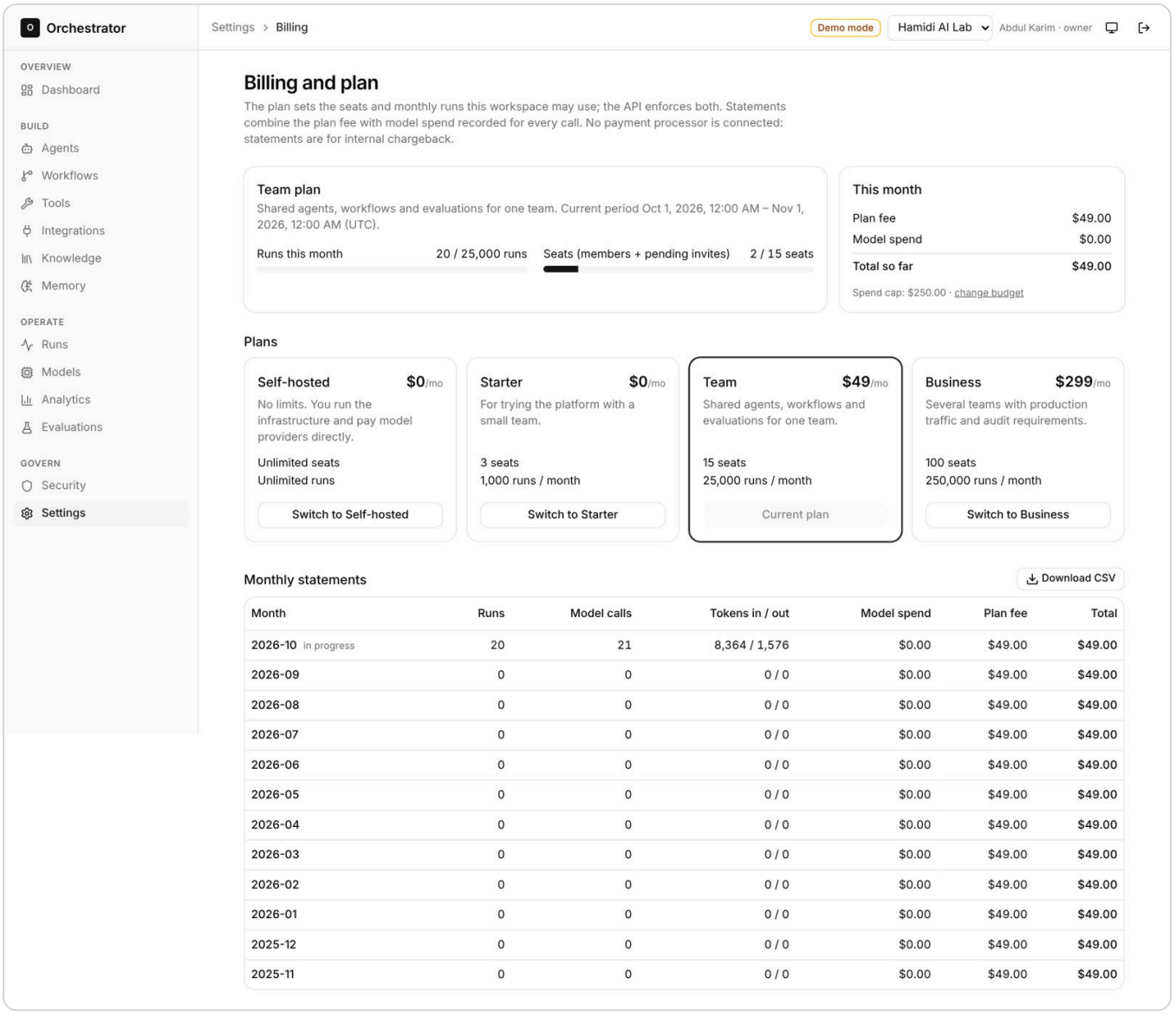
42. API keys `/settings/api-keys`

Workspace API keys with a role cap and an expiry, so your own applications can call agents and workflows (Authorization: Bearer orch_...).



43. Billing and plan (new) `/settings/billing`

Plans set seat and monthly run quotas, and the API enforces both. Usage meters, monthly statements (plan fee plus model spend) and CSV export. No payment processor is connected; the statements are for internal chargeback.



44. API documentation `/api/docs`

Interactive Swagger documentation of the REST API, including the new /workflows, /eval-datasets, /eval-runs, /memories, /billing and /analytics endpoints.

Running the project

Docker: `./scripts/dev.sh` (Windows: `./scripts/dev.ps1`) builds and starts the API, worker and web app. Then open `http://localhost:3000`. The API docs are at `http://localhost:8000/api/docs`.

Without Docker: `./scripts/dev.sh --local` (needs uv and Node.js 20.9+).

Database: `.env` contains `MONGODB_URI` and `COMPOSE_MONGODB_URI` for the Atlas cluster, and `MONGODB_DB=AI`. In Atlas, under Network Access, allow the IP address of every machine that runs the API and the worker.

First steps: register, open Agents, choose New agent with provider "Demo", publish it, try it in the Test console, then build a workflow around it on the Workflows page.